9. Modul 07 — IEEE 802.15.4 Raw Frame (P2P)

IoT COMMUNICATION LAB · MODUL 07

Speak Raw 802.15.4 — IEEE 802.15.4 Raw Frame (P2P)

ESP32-H2 · 802.15.4 · raw MAC frame · level Intermediate · 3 × 50 menit · folder kode `week07_802154_p2p`

PENGANTAR MODUL · Gambaran Umum

Modul 07 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya menyusun sendiri frame MAC 802.15.4 byte demi byte dan membuatnya terbang tanpa bantuan stack apa pun. Percobaan berjalan sebagai pertukaran raw frame PING/PONG antara dua node, diamati melalui dua terminal Serial Monitor pada 115200 baud.

9.1 Pendahuluan

Enam modul pertama berjalan di atas BLE, tempat stack menyembunyikan semua detail radio. Modul ini **membuka lantai dasar**: tidak ada Zigbee, tidak ada Thread, tidak ada GATT — hanya PHY/MAC 802.15.4 dan array byte yang disusun sendiri. Setelah modul ini, setiap kali Zigbee (M08–M10) atau Thread (M11–M13) tampak "langsung bekerja", praktikan mengetahui persis apa yang sebenarnya dikerjakan protokol tersebut.

Prasyaratnya adalah M01–M06: alur build, pembacaan Serial Monitor sebagai instrumen, dan pengukuran loss. Yang dibangun di sini adalah pemahaman struktur MHR 802.15.4, pengaturan channel dan PAN ID, short address, penyusunan frame secara manual, serta penanganan callback penerimaan di konteks ISR. Semuanya dipakai lagi pada M08–M10 karena Zigbee memakai PHY/MAC yang sama, M11–M13 karena Thread pun demikian, dan M16 saat overhead antarprotokol dibandingkan di atas radio yang identik.

Peta modul — titik balik seri ini

Modul	Lapisan yang dikerjakan
01–06	BLE — stack menyembunyikan radio
07 (ini)	**802.15.4 telanjang — frame disusun sendiri**
08–10	Zigbee di atas 802.15.4 — stack mengurus join, binding, routing
11–13	Thread di atas 802.15.4 — stack mengurus IPv6, mesh, dataset

Tabel 9.1: Titik balik lapisan pada Modul 07

Kontrak data lab ini. Radio yang dipakai M07–M13 **sama persis** (IEEE 802.15.4, channel 15). Yang berbeda hanya lapisan di atasnya. Karena itu angka RSSI dan jangkauan yang diukur pada modul ini dapat dipakai sebagai garis dasar (*baseline*) saat membandingkan Zigbee dan Thread di M16.

9.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Menyusun dan menerbangkan frame MAC 802.15.4 sendiri

1. Menyusun frame MAC 802.15.4 lengkap (Len ditambah MHR 11 byte ditambah payload) dan menunjukkan letak tiap field pada dump heksadesimal frame yang benar-benar tertangkap.
2. Menjelaskan tiga aturan API `esp_ieee802154_*` yang wajib dipatuhi (panjang `Len` termasuk FCS, `esp_ieee802154_receive()` di `setup()`, buffer TX harus `static`) beserta gejala kegagalannya.
3. Menjalankan pertukaran PING/PONG dua arah dan menghitung packet loss tiap arah secara terpisah.
4. Membuktikan pengaruh channel dan PAN ID terhadap keterhubungan dengan mengubahnya dan mengamati akibatnya.
5. Membuktikan bahwa penyaringan PAN ID dan alamat tujuan hanya aktif bila promiscuous mode dimatikan dengan `esp_ieee802154_set_promiscuous(false)`.

Kriteria keberhasilan

- ☐ Setiap `PING n` dibalas `PONG n` pada jarak dekat, isi payload utuh (bukan karakter acak).
- ☐ Komunikasi terbukti berhenti saat channel salah satu node dibedakan.
- ☐ PAN ID berbeda terbukti tersaring hanya bila promiscuous mode dimatikan (EXP-05).
- ☐ Loss tiap arah terukur terpisah (PING hilang dibanding PONG hilang).
- ☐ Dump frame heksadesimal dianalisis dan tiap field ditunjuk.

9.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (modulasi O-QPSK, CSMA-CA, superframe, beacon) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 9.2.

Istilah	Definisi kerja di lab ini
IEEE 802.15.4	Standar PHY/MAC nirkabel low-rate low-power; basis Zigbee dan Thread.
Channel	Frekuensi kerja radio (di sini 15); kedua node harus sama.
PAN ID	Identitas jaringan (`0xCAFE`); frame di luar PAN disaring hardware — **hanya bila promiscuous mode dimatikan**.
Promiscuous mode	Mode "dengar semua": filter PAN ID dan alamat tujuan di MAC dimatikan, setiap frame dengan FCS valid diteruskan ke callback. Driver ESP-IDF menyalakannya **secara default**, sehingga kode memanggil `esp_ieee802154_set_promiscuous(false)` di `setup()`.
Short address	Alamat 16-bit node (`0x0001` atau `0x0002`).
Frame / MHR	Header MAC `[Len][FC(2)][Seq(1)][DestPAN(2)][DestAddr(2)][SrcPAN(2)][SrcAddr(2)][payload]`.
Alamat pengirim	Dibaca callback dari `SrcAddr` (`frame[10..11]`, little-endian) — setara info alamat asal pada frame RX mode API XBee. Log `RX dari 0x....` mencetak alamat ini; bila frame bukan untuk node tersebut (broadcast atau promiscuous), tujuan dan PAN-nya ikut dicetak: `RX dari 0x0001 (ke 0x0002, PAN 0xCAFE)`. Node2 dan Node3 membalas ke alamat ini dan **hanya membalas PING**.
RSSI / LQI	Disalin callback dari `frame_info->rssi` dan `frame_info->lqi`, dicetak di akhir baris RX: `RX dari 0x0002: PONG 1  [RSSI -12 dBm, LQI 11]`. RSSI (dBm) sudah dikoreksi driver ESP-IDF. LQI adalah nilai mentah hardware ESP32-H2 dengan skala khusus chip (teramati 6–11), **tidak** sebanding dengan LQI 0–255 radio lain — untuk pengukuran jarak pakai RSSI.
FCS	Checksum 2 byte; **isinya** dihitung hardware, tetapi **panjangnya tetap ikut** pada byte `Len`.

Istilah	Definisi kerja di lab ini
RX when idle	Radio kembali ke RX setiap selesai TX atau RX — bukan pengganti `esp_ieee802154_receive()`.
ISR	`esp_ieee802154_receive_done()` berjalan di konteks interupsi: salin data, set flag, jangan mencetak.

Tabel 9.2: Istilah kerja Modul 07

Tiga jebakan API `esp_ieee802154_*`. Ketiganya nyata dan ditemukan saat modul ini diuji di perangkat. Kode yang disediakan sudah memperhitungkannya — yang dituntut di sini adalah kemampuan menjelaskan gejalanya (Tabel 9.3).

Aturan	Gejala bila dilanggar
`Len` = MHR + payload + 2 (FCS)	Frame ditolak atau payload terpotong di penerima
`esp_ieee802154_receive()` wajib dipanggil di `setup()`	TX jalan, tetapi tidak ada satu pun frame diterima
Buffer TX harus tetap hidup sampai transmisi selesai (`static`, bukan variabel lokal)	Payload sesekali berubah menjadi sampah atau isi RAM lama

Tabel 9.3: Tiga aturan API `esp_ieee802154_*` dan gejala pelanggarannya

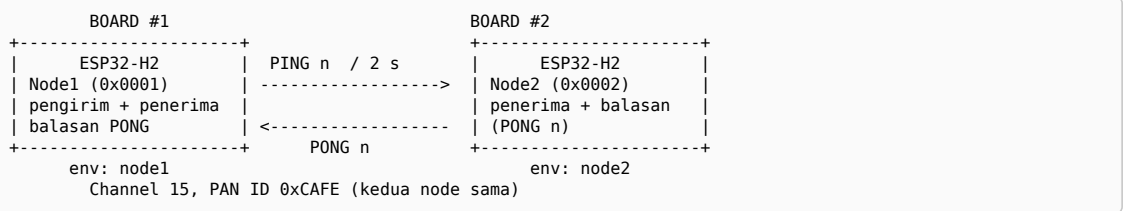
Aturan ketiga paling menipu: `esp_ieee802154_transmit()` bersifat **asinkron** dan hanya menyimpan *pointer* ke buffer. Bila buffer dideklarasikan sebagai variabel lokal di `loop()`, isinya sudah tertimpa stack frame lain saat radio benar-benar mengirim — sebagian frame berangkat berisi sampah, sebagian lain kebetulan masih utuh. Itulah mengapa gejalanya *intermiten*, bukan gagal total.

Catatan — filter PAN tidak aktif dengan sendirinya. `esp_ieee802154_set_panid()` dan `esp_ieee802154_set_short_address()` hanya mengisi register; nilai itu baru dipakai untuk menyaring bila promiscuous mode mati. Karena driver ESP-IDF menyalakan promiscuous secara default, tanpa `esp_ieee802154_set_promiscuous(false)` dua node dengan PAN ID berbeda tetap bisa saling bertukar pesan, dan node lain ikut menerima unicast yang bukan untuknya. Perilaku ini berbeda dengan XBee, yang firmware-nya selalu menyaring PAN ID dan alamat tujuan. Pengaruh baris ini dibuktikan pada EXP-05.

Sekuens protokol yang diamati

```
buildFrame()                        radio 802.15.4
[Len][FC][Seq][DPAN][DA][SPAN][SA][payload]
       |                                  |
       ▼                                  ▼
 esp_ieee802154_transmit() ─► udara ─► esp_ieee802154_receive_done() (ISR)
                                                  |
                                 flag hasRx ──► loop(): cetak "RX dari ..."
```

9.4 Topologi



Node	Board	Environment	Short addr	Aksi
Node1	ESP32-H2 DevKitM-1	`node1`	`0x0001`	TX `PING n` tiap 2 s, RX balasan
Node2	ESP32-H2 DevKitM-1	`node2`	`0x0002`	RX `PING n`, TX `PONG n`

Tabel 9.4: Peran tiap node Modul 07

Radio 802.15.4 dipakai **telanjang** (tanpa Zigbee atau Thread) di dua **ESP32-H2 DevKitM-1**. ESP32-C6 juga punya radio 802.15.4 dan bisa dipakai, tetapi lab ini menyimpannya untuk peran gateway Wi-Fi di Modul 13.

9.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan API ESP-IDF `esp_ieee802154.h`.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1	2
2	Kabel USB data	kabel data, bukan *charge-only*	2
3	PC/Laptop	PlatformIO Core/IDE, 2 port USB bebas	1
4	Platform PlatformIO	pioarduino `espressif32` 55.03.311 (API `esp_ieee802154.h` bawaan)	—

Tabel 9.5: Alat dan bahan Modul 07

Tidak ada library eksternal — modul ini memanggil API ESP-IDF langsung.

Radio config

Parameter	Nilai
Channel	15
PAN ID	`0xCAFE`
Short address	Node1 `0x0001`, Node2 `0x0002`
Interval PING	2000 ms

Tabel 9.6: Konfigurasi radio Modul 07

9.6 Kode Program

Pin port agar tidak salah flash (Kode 9.1).

```
[env:node1]
build_src_filter = +<node1/*.cpp>
upload_port  = /dev/ttyACM0
monitor_port = /dev/ttyACM0

[env:node2]
build_src_filter = +<node2/*.cpp>
upload_port  = /dev/ttyACM2
monitor_port = /dev/ttyACM2

[env:node3]
build_src_filter = +<node3/*.cpp>
; port diisi sesuai board ketiga (opsional, untuk EXP-04-e broadcast)
```

Kode 9.1: Potongan `platformio.ini` dengan port yang dipin per environment

```
[platformio]
default_envs = node1

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200

[env:node1]
build_src_filter = +<node1/*.cpp>

[env:node2]
build_src_filter = +<node2/*.cpp>

[env:node3]
build_src_filter = +<node3/*.cpp>
```

Kode 9.2: `platformio.ini` Modul 07 pada repositori

```
// Minggu 7 — IEEE 802.15.4 P2P (raw frame), Node1 = pengirim (short addr 0x0001)
#include <Arduino.h>
#include "esp_ieee802154.h"

#define CHANNEL   15
#define PAN_ID    0xCAFE
#define MY_ADDR   0x0001
#define PEER_ADDR 0x0002
#define MHR_LEN   11   // FC(2)+Seq(1)+DestPAN(2)+DestAddr(2)+SrcPAN(2)+SrcAddr(2)
#define FCS_LEN   2    // ikut dihitung pada byte Len; saat RX diganti RSSI+LQI

// Buffer hasil terima (diisi ISR, dibaca loop)
static volatile bool hasRx = false;
static char rxPayload[64] = {0};
static volatile uint16_t rxSrc, rxDst, rxDstPan;  // dibaca dari MHR frame yang diterima
static volatile int8_t  rxRssi;                   // kekuatan sinyal (dBm, sudah dikoreksi driver)
static volatile uint8_t rxLqi;                    // LQI mentah hardware (skala khusus chip)

// Susun frame: [Len][FC(2)][Seq(1)][DestPAN(2)][DestAddr(2)][SrcPAN(2)][SrcAddr(2)][payload]
// FCS dihitung hardware otomatis, tetapi 2 byte-nya tetap ikut dihitung pada "Len".
static uint8_t buildFrame(uint8_t *frame, uint16_t dst, const char *payload) {
  uint8_t plen = strlen(payload);
  uint8_t mhr = 11;                       // 2+1+2+2+2+2
  uint8_t len = mhr + plen + FCS_LEN;
  frame[0] = len;                         // byte "Len" (PHR), termasuk FCS
  frame[1] = 0x01; frame[2] = 0x88;       // Frame Control (data, addr 16-bit)
  frame[3] = 0x00;                        // sequence number
  frame[4] = PAN_ID & 0xFF; frame[5] = (PAN_ID >> 8) & 0xFF;       // dest PAN
  frame[6] = dst & 0xFF;    frame[7] = (dst >> 8) & 0xFF;          // dest addr
  frame[8] = PAN_ID & 0xFF; frame[9] = (PAN_ID >> 8) & 0xFF;       // src PAN
  frame[10] = MY_ADDR & 0xFF; frame[11] = (MY_ADDR >> 8) & 0xFF;   // src addr
  memcpy(&frame[12], payload, plen);
  return 1 + len;  // total byte dalam buffer
}

// ISR: frame diterima. [Len]...[MHR]...[payload][RSSI][LQI]
void esp_ieee802154_receive_done(uint8_t *frame, esp_ieee802154_frame_info_t *frame_info) {
  uint8_t len = frame[0];
  uint8_t plen = len - MHR_LEN - FCS_LEN;
  // Offset tetap hanya berlaku untuk format buildFrame(): FC 0x8801 (data, addr 16-bit)
  if (frame[1] == 0x01 && frame[2] == 0x88 && plen > 0 && plen < sizeof(rxPayload)) {
    rxDstPan = frame[4]  | (frame[5]  << 8);   // dest PAN
    rxDst    = frame[6]  | (frame[7]  << 8);   // dest addr
    rxSrc    = frame[10] | (frame[11] << 8);   // src addr: alamat pengirim sebenarnya
    rxRssi   = frame_info->rssi;
    rxLqi    = frame_info->lqi;
    memcpy((void *)rxPayload, &frame[12], plen);
    rxPayload[plen] = '\0';
    hasRx = true;
  }
  esp_ieee802154_receive_handle_done(frame);
}

// Cetak alamat pengirim dari header frame. Tujuan/PAN ikut dicetak bila frame
// bukan untuk node ini (broadcast, atau lolos karena promiscuous aktif).
static void printRx(uint16_t src, uint16_t dst, uint16_t dpan) {
```

```
  if (dst == MY_ADDR && dpan == PAN_ID)
    Serial.printf("RX dari 0x%04X: %s  [RSSI %d dBm, LQI %u]\n",
                  src, rxPayload, rxRssi, rxLqi);
  else
    Serial.printf("RX dari 0x%04X (ke 0x%04X, PAN 0x%04X): %s  [RSSI %d dBm, LQI %u]\n",
                  src, dst, dpan, rxPayload, rxRssi, rxLqi);
}

void setup() {
  Serial.begin(115200);
  Serial.println("Node1 (802.15.4 sender) starting...");

  uint8_t ext[] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08};
  esp_ieee802154_enable();
  esp_ieee802154_set_channel(CHANNEL);
  esp_ieee802154_set_panid(PAN_ID);
  esp_ieee802154_set_short_address(MY_ADDR);
  esp_ieee802154_set_extended_address(ext);
  esp_ieee802154_set_promiscuous(false);  // aktifkan filter PAN ID & alamat di hardware (default driver:
true)
  esp_ieee802154_set_rx_when_idle(true);
  esp_ieee802154_receive();  // masuk state RX; tanpa ini radio diam di idle

  Serial.printf("Channel %d, PAN 0x%04X, short addr 0x%04X\n", CHANNEL, PAN_ID, MY_ADDR);
}

void loop() {
  if (hasRx) {
    hasRx = false;
    printRx(rxSrc, rxDst, rxDstPan);
  }

  static unsigned long last = 0;
  static uint32_t count = 0;
  if (millis() - last > 2000) {
    last = millis();
    char msg[32];
    snprintf(msg, sizeof(msg), "PING %lu", (unsigned long)++count);
    // static: driver 802.15.4 mengirim secara asinkron, buffer harus tetap
    // hidup sampai transmisi selesai (buffer di stack akan tertimpa)
    static uint8_t frame[128];
    uint8_t total = buildFrame(frame, PEER_ADDR, msg);
    esp_ieee802154_transmit(frame, true);
    Serial.printf("TX ke 0x%04X: %s\n", PEER_ADDR, msg);
  }
}
```

Kode 9.3: `src/node1/main.cpp` — pengirim PING `0x0001`

```
// Minggu 7 — IEEE 802.15.4 P2P (raw frame), Node2 = penerima (short addr 0x0002)
#include <Arduino.h>
#include "esp_ieee802154.h"

#define CHANNEL   15
#define PAN_ID    0xCAFE
#define MY_ADDR   0x0002
#define MHR_LEN   11   // FC(2)+Seq(1)+DestPAN(2)+DestAddr(2)+SrcPAN(2)+SrcAddr(2)
#define FCS_LEN   2    // ikut dihitung pada byte Len; saat RX diganti RSSI+LQI

static volatile bool hasRx = false;
static char rxPayload[64] = {0};
static volatile uint16_t rxSrc, rxDst, rxDstPan;  // dibaca dari MHR frame yang diterima
static volatile int8_t  rxRssi;                   // kekuatan sinyal (dBm, sudah dikoreksi driver)
static volatile uint8_t rxLqi;                    // LQI mentah hardware (skala khusus chip)

static uint8_t buildFrame(uint8_t *frame, uint16_t dst, const char *payload) {
  uint8_t plen = strlen(payload);
  uint8_t mhr = MHR_LEN;
  uint8_t len = mhr + plen + FCS_LEN;
  frame[0] = len;   // byte Len termasuk FCS
  frame[1] = 0x01; frame[2] = 0x88;
  frame[3] = 0x00;
  frame[4] = PAN_ID & 0xFF; frame[5] = (PAN_ID >> 8) & 0xFF;
  frame[6] = dst & 0xFF;    frame[7] = (dst >> 8) & 0xFF;
  frame[8] = PAN_ID & 0xFF; frame[9] = (PAN_ID >> 8) & 0xFF;
  frame[10] = MY_ADDR & 0xFF; frame[11] = (MY_ADDR >> 8) & 0xFF;
  memcpy(&frame[12], payload, plen);
  return 1 + len;
```

```
}

void esp_ieee802154_receive_done(uint8_t *frame, esp_ieee802154_frame_info_t *frame_info) {
  uint8_t len = frame[0];
  uint8_t plen = len - MHR_LEN - FCS_LEN;
  // Offset tetap hanya berlaku untuk format buildFrame(): FC 0x8801 (data, addr 16-bit)
  if (frame[1] == 0x01 && frame[2] == 0x88 && plen > 0 && plen < sizeof(rxPayload)) {
    rxDstPan = frame[4]  | (frame[5]  << 8);   // dest PAN
    rxDst    = frame[6]  | (frame[7]  << 8);   // dest addr
    rxSrc    = frame[10] | (frame[11] << 8);   // src addr: alamat pengirim sebenarnya
    rxRssi   = frame_info->rssi;
    rxLqi    = frame_info->lqi;
    memcpy((void *)rxPayload, &frame[12], plen);
    rxPayload[plen] = '\0';
    hasRx = true;
  }
  esp_ieee802154_receive_handle_done(frame);
}

// Cetak alamat pengirim dari header frame. Tujuan/PAN ikut dicetak bila frame
// bukan untuk node ini (broadcast, atau lolos karena promiscuous aktif).
static void printRx(uint16_t src, uint16_t dst, uint16_t dpan) {
  if (dst == MY_ADDR && dpan == PAN_ID)
    Serial.printf("RX dari 0x%04X: %s  [RSSI %d dBm, LQI %u]\n",
                  src, rxPayload, rxRssi, rxLqi);
  else
    Serial.printf("RX dari 0x%04X (ke 0x%04X, PAN 0x%04X): %s  [RSSI %d dBm, LQI %u]\n",
                  src, dst, dpan, rxPayload, rxRssi, rxLqi);
}

void setup() {
  Serial.begin(115200);
  Serial.println("Node2 (802.15.4 receiver) starting...");

  uint8_t ext[] = {0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18};
  esp_ieee802154_enable();
  esp_ieee802154_set_channel(CHANNEL);
  esp_ieee802154_set_panid(PAN_ID);
  esp_ieee802154_set_short_address(MY_ADDR);
  esp_ieee802154_set_extended_address(ext);
  esp_ieee802154_set_promiscuous(false);  // aktifkan filter PAN ID & alamat di hardware (default driver:
true)
  esp_ieee802154_set_rx_when_idle(true);
  esp_ieee802154_receive();  // masuk state RX; tanpa ini radio diam di idle

  Serial.printf("Channel %d, PAN 0x%04X, short addr 0x%04X\n", CHANNEL, PAN_ID, MY_ADDR);
}

void loop() {
  if (hasRx) {
    hasRx = false;
    uint16_t src = rxSrc;
    printRx(src, rxDst, rxDstPan);

    // Hanya PING yang dibalas; PONG milik node lain (tertangkap saat
    // promiscuous) diabaikan agar tidak terjadi balas-membalas tanpa henti
    if (strncmp(rxPayload, "PING ", 5) != 0) return;

    // Balas PONG ke alamat pengirim sebenarnya
    char msg[32];
    snprintf(msg, sizeof(msg), "PONG %s", rxPayload + 5);  // hilangkan "PING "
    // static: driver 802.15.4 mengirim secara asinkron, buffer harus tetap
    // hidup sampai transmisi selesai (buffer di stack akan tertimpa)
    static uint8_t frame[128];
    uint8_t total = buildFrame(frame, src, msg);
    esp_ieee802154_transmit(frame, true);
    Serial.printf("TX balasan ke 0x%04X: %s\n", src, msg);
  }
}
```

Kode 9.4: `src/node2/main.cpp` — penerima dan pembalas PONG `0x0002`

Berkas `src/node3/main.cpp` (Kode 9.5) adalah node ketiga bershort address `0x0003` yang dipakai pada uji broadcast EXP-04-e. Node2 dan Node3 tidak memakai `PEER_ADDR`: keduanya membalas ke alamat pengirim yang dibaca dari `SrcAddr` frame, dan hanya membalas frame berisi PING.

```
// Minggu 7 — IEEE 802.15.4 P2P (raw frame), Node3 = penerima tambahan (short addr 0x0003)
// Dipakai untuk uji broadcast (EXP-04-e): menerima frame yang dikirim ke 0xFFFF
#include <Arduino.h>
#include "esp_ieee802154.h"

#define CHANNEL   15
#define PAN_ID    0xCAFE
#define MY_ADDR   0x0003
#define MHR_LEN   11   // FC(2)+Seq(1)+DestPAN(2)+DestAddr(2)+SrcPAN(2)+SrcAddr(2)
#define FCS_LEN   2    // ikut dihitung pada byte Len; saat RX diganti RSSI+LQI

static volatile bool hasRx = false;
static char rxPayload[64] = {0};
static volatile uint16_t rxSrc, rxDst, rxDstPan;  // dibaca dari MHR frame yang diterima
static volatile int8_t  rxRssi;                   // kekuatan sinyal (dBm, sudah dikoreksi driver)
static volatile uint8_t rxLqi;                    // LQI mentah hardware (skala khusus chip)

static uint8_t buildFrame(uint8_t *frame, uint16_t dst, const char *payload) {
  uint8_t plen = strlen(payload);
  uint8_t mhr = MHR_LEN;
  uint8_t len = mhr + plen + FCS_LEN;
  frame[0] = len;   // byte Len termasuk FCS
  frame[1] = 0x01; frame[2] = 0x88;
  frame[3] = 0x00;
  frame[4] = PAN_ID & 0xFF; frame[5] = (PAN_ID >> 8) & 0xFF;
  frame[6] = dst & 0xFF;    frame[7] = (dst >> 8) & 0xFF;
  frame[8] = PAN_ID & 0xFF; frame[9] = (PAN_ID >> 8) & 0xFF;
  frame[10] = MY_ADDR & 0xFF; frame[11] = (MY_ADDR >> 8) & 0xFF;
  memcpy(&frame[12], payload, plen);
  return 1 + len;
}

void esp_ieee802154_receive_done(uint8_t *frame, esp_ieee802154_frame_info_t *frame_info) {
  uint8_t len = frame[0];
  uint8_t plen = len - MHR_LEN - FCS_LEN;
  // Offset tetap hanya berlaku untuk format buildFrame(): FC 0x8801 (data, addr 16-bit)
  if (frame[1] == 0x01 && frame[2] == 0x88 && plen > 0 && plen < sizeof(rxPayload)) {
    rxDstPan = frame[4]  | (frame[5]  << 8);   // dest PAN
    rxDst    = frame[6]  | (frame[7]  << 8);   // dest addr
    rxSrc    = frame[10] | (frame[11] << 8);   // src addr: alamat pengirim sebenarnya
    rxRssi   = frame_info->rssi;
    rxLqi    = frame_info->lqi;
    memcpy((void *)rxPayload, &frame[12], plen);
    rxPayload[plen] = '\0';
    hasRx = true;
  }
  esp_ieee802154_receive_handle_done(frame);
}

// Cetak alamat pengirim dari header frame. Tujuan/PAN ikut dicetak bila frame
// bukan untuk node ini (broadcast, atau lolos karena promiscuous aktif).
static void printRx(uint16_t src, uint16_t dst, uint16_t dpan) {
  if (dst == MY_ADDR && dpan == PAN_ID)
    Serial.printf("RX dari 0x%04X: %s  [RSSI %d dBm, LQI %u]\n",
                  src, rxPayload, rxRssi, rxLqi);
  else
    Serial.printf("RX dari 0x%04X (ke 0x%04X, PAN 0x%04X): %s  [RSSI %d dBm, LQI %u]\n",
                  src, dst, dpan, rxPayload, rxRssi, rxLqi);
}

void setup() {
  Serial.begin(115200);
  Serial.println("Node3 (802.15.4 receiver) starting...");

  uint8_t ext[] = {0x21, 0x22, 0x23, 0x24, 0x25, 0x26, 0x27, 0x28};
  esp_ieee802154_enable();
  esp_ieee802154_set_channel(CHANNEL);
  esp_ieee802154_set_panid(PAN_ID);
  esp_ieee802154_set_short_address(MY_ADDR);
  esp_ieee802154_set_extended_address(ext);
  esp_ieee802154_set_promiscuous(false);  // aktifkan filter PAN ID & alamat di hardware (default driver:
true)
  esp_ieee802154_set_rx_when_idle(true);
  esp_ieee802154_receive();  // masuk state RX; tanpa ini radio diam di idle

  Serial.printf("Channel %d, PAN 0x%04X, short addr 0x%04X\n", CHANNEL, PAN_ID, MY_ADDR);
}

void loop() {
  if (hasRx) {
    hasRx = false;
    uint16_t src = rxSrc;
```

```
    printRx(src, rxDst, rxDstPan);

    // Hanya PING yang dibalas; PONG milik node lain (tertangkap saat
    // promiscuous) diabaikan agar tidak terjadi balas-membalas tanpa henti
    if (strncmp(rxPayload, "PING ", 5) != 0) return;

    // Balas PONG ke alamat pengirim sebenarnya
    char msg[32];
    snprintf(msg, sizeof(msg), "PONG %s", rxPayload + 5);  // hilangkan "PING "
    static uint8_t frame[128];
    uint8_t total = buildFrame(frame, src, msg);
    esp_ieee802154_transmit(frame, true);
    Serial.printf("TX balasan ke 0x%04X: %s\n", src, msg);
  }
}
```

Kode 9.5: `src/node3/main.cpp` — node ketiga `0x0003` untuk uji broadcast

9.7 Build dan Flash

```
pio device list
pio run -d week07_802154_p2p -e node1 -t upload
pio run -d week07_802154_p2p -e node2 -t upload -t monitor
```

Memantau semua node dari satu komputer. `pio device monitor` hanya membuka satu port. Skrip `monitor_serial.py` (ada di folder `week07_802154_p2p` pada repositori) membuka semua port UART CH343 sekaligus, menampilkan ketiga node dalam satu jendela dengan timestamp bersama, lalu mencetak ringkasan per link (terkirim, diterima, loss, RSSI, LQI) saat berhenti.

```
python week07_802154_p2p/monitor_serial.py                    # deteksi port otomatis
python week07_802154_p2p/monitor_serial.py --duration 120 --log sesi1.txt
python week07_802154_p2p/monitor_serial.py --port COM5 --port COM11 --port COM13
python3 week07_802154_p2p/monitor_serial.py --port /dev/ttyACM0 --port /dev/ttyACM2
```

Nama node dikenali dari banner saat boot, jadi urutan port tidak perlu diingat. Tiap board di-reset sekali saat port dibuka agar hitungan PING dimulai dari 1 (pakai `--no-reset` untuk mengamati tanpa reset). Tutup dulu `pio device monitor` — satu port tidak bisa dibuka dua program sekaligus. Contoh ringkasan pada baseline:

```
Link (pengirim -> penerima)      jenis  kirim  terima   loss   RSSI dBm rata2 (min..max)
-------------------------------------------------------------------------------------
Node1(0x0001) -> Node2(0x0002)   PING       8      8    0.0%    -11.0 (-11..-11)
Node2(0x0002) -> Node1(0x0001)   PONG       8      8    0.0%    -12.0 (-12..-12)
```

PENTING

Pilih port USB-to-UART, bukan USB native. Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- ☐ `pio device list` dijalankan, dua port dicatat dan diisikan di atas.
- ☐ Dua ESP32-H2 terpasang dan terdeteksi.
- ☐ Environment `node1` dan `node2` dikenali PlatformIO.
- ☐ Serial Monitor 115200 baud untuk kedua node.
- ☐ Tidak ada firmware Zigbee atau Thread yang masih berjalan di board yang sama (radio 802.15.4 dipakai langsung).

9.8 Percobaan

9.8.1 EXP-01 — Konfigurasi Radio dan Anatomi Frame

Unggah kedua firmware, verifikasi konfigurasi radio pada baris pertama Serial Monitor, lalu telusuri fungsi `buildFrame()`.

```
frame[0]=Len | [1..2]=FC 0x8801 | [3]=Seq | [4..5]=PAN 0xCAFE
[6..7]=DestAddr | [8..9]=PAN | [10..11]=SrcAddr | [12..]=payload

Len = 11 (MHR) + panjang payload + 2 (FCS)
```

Contoh frame `PING 38` yang benar-benar tertangkap di udara (dump `receive_done` pada ESP32-H2; dua byte terakhir adalah RSSI dan LQI yang menggantikan FCS saat penerimaan).

```
14 01 88 00 FE CA 02 00 FE CA 01 00 50 49 4E 47 20 33 38 E5 0A
|  |     |  |     |     |     |     "P  I  N  G  _  3  8" |  |
|  FC    |  PAN   Dest  PAN   Src                          |  LQI
Len=0x14 Seq      0x0002      0x0001                       RSSI
```

Data capture

Parameter	Hasil
Channel	
PAN ID	
Short address Node1	
Short address Node2	
Ukuran MHR (byte)	
`Len` untuk payload 7 karakter	

Tabel 9.7: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Hitung sendiri nilai `Len` untuk payload `"PING 38"` (7 karakter) dan cocokkan dengan `0x14` pada dump di atas. Lalu jawab: jika FCS dihitung hardware, mengapa panjangnya tetap harus disertakan? Petunjuk: `Len` adalah field PHY, dan PHY menghitung **semua** byte yang mengudara.

CHECKPOINT

Kedua node mencetak baris `Channel 15, PAN 0xCAFE, short addr 0x000X` dengan nilai yang berbeda untuk tiap node. Jika keduanya mencetak alamat yang sama, berarti terjadi kesalahan flash — periksa `upload_port`.

9.8.2 EXP-02 — Pertukaran PING dan PONG

Node1 mengirim `PING n` tiap 2 s; Node2 menerima (callback `esp_ieee802154_receive_done` menyalin payload dan menyetel flag, `loop()` mencetak), lalu membalas `PONG n`; Node1 menerima balasan itu.

```
Node1 loop (tiap 2 s)            Node2 receive_done
TX "PING n" ──────────────────►  RX dari 0x0001: PING n
RX dari 0x0002: PONG n  ◄──────  TX balasan "PONG n"
```

Expected output — Node1

```
Node1 (802.15.4 sender) starting...
Channel 15, PAN 0xCAFE, short addr 0x0001
TX ke 0x0002: PING 1
RX dari 0x0002: PONG 1  [RSSI -12 dBm, LQI 11]
TX ke 0x0002: PING 2
RX dari 0x0002: PONG 2  [RSSI -12 dBm, LQI 11]
```

Expected output — Node2

```
Node2 (802.15.4 receiver) starting...
Channel 15, PAN 0xCAFE, short addr 0x0002
RX dari 0x0001: PING 1  [RSSI -11 dBm, LQI 10]
TX balasan ke 0x0001: PONG 1
```

Nilai RSSI dan LQI bergantung pada jarak dan posisi board; angka di atas diambil dengan board berdekatan di meja.

CHECKPOINT

Isi payload harus **terbaca sebagai teks**, bukan karakter acak. Jika muncul sampah tak terbaca, itu gejala buffer TX bukan `static` (lihat bagian Dasar Teori). Jika Node2 hanya mencetak baris konfigurasi dan tidak pernah `RX`, `esp_ieee802154_receive()` tidak dipanggil. Perbaiki dulu, jangan lanjut mengukur.

9.8.3 EXP-03 — Isolasi Channel dan RTT

1. Ubah jarak antar node secara bertahap.
2. Ubah `CHANNEL` pada **salah satu** node (perlu unggah ulang) sehingga berbeda, lalu verifikasi komunikasi berhenti total.
3. Kembalikan ke channel yang sama, lalu ubah `PAN_ID` salah satu node — amati apakah gejalanya sama atau berbeda dengan kasus channel.
4. Ukur round-trip kasar PING ke PONG dari waktu di Serial Monitor.

Data capture

Parameter	Hasil
Perilaku saat channel berbeda	
Perilaku saat PAN ID berbeda	
Round-trip PING ke PONG (kasar)	
Interval kirim PING (s)	
Pesan per menit yang dibalas	

Tabel 9.8: Lembar pengamatan EXP-03

CHECKPOINT

Praktikan dapat menjelaskan **perbedaan** antara gagal karena channel dan gagal karena PAN ID (petunjuk: yang satu radio tidak mendengar sama sekali, yang satu mendengar tetapi menyaring). Ini penting untuk memahami penyaringan Zigbee dan Thread di modul berikutnya.

9.8.4 EXP-04 — Efek Parameter: PAN ID, Channel, Broadcast

Semua efek di bawah cukup dicapai dengan **mengubah `#define` di `src/nodeX/main.cpp`** — tidak ada logika yang diubah. Tiap ubah parameter, unggah ulang node yang bersangkutan.

Uji	`node1`	`node2`	Gejala yang harus terlihat
04-a PAN sama	`PAN_ID 0xCAFE`	`PAN_ID 0xCAFE`	PING–PONG normal (baseline)
04-b PAN beda	`PAN_ID 0xCAFE`	`PAN_ID 0xBEEF`	Node1 tetap cetak `TX`, Node2 **tidak pernah** cetak `RX` — frame *heard but filtered* (disaring hardware MAC)
04-c Channel sama	`CHANNEL 15`	`CHANNEL 15`	PING–PONG normal
04-d Channel beda	`CHANNEL 15`	`CHANNEL 20`	Kedua node sunyi di sisi RX — radio *never heard* (tuli, tidak mendengar sama sekali)
04-e Broadcast	`PEER_ADDR 0xFFFF`	tidak diubah	Node2 **dan** Node3 menerima frame yang sama (`RX dari 0x0001 (ke 0xFFFF, ...)`); keduanya membalas bersamaan sehingga Node1 hanya menerima PONG dari **salah satu** node

Tabel 9.9: Matriks uji efek parameter EXP-04

Perbedaan 04-b dengan 04-d adalah inti yang harus bisa dijelaskan.

PAN ID beda (04-b) radio memang menerima transmisi di channel yang sama, tetapi MAC hardware **menyaring frame** dengan PAN ID asing sebelum sampai ke callback (berlaku karena `setup()` mematikan promiscuous mode; lihat EXP-05). TX di sisi lain tetap berjalan normal — komunikasi terlihat "searah hilang".

Channel beda (04-d) radio tidak berada di frekuensi yang sama, sehingga tidak ada yang diterima secara fisik. Tidak ada filter yang "menolak" karena memang tidak ada yang masuk.

Uji broadcast (04-e) — wajib 3 node. Firmware node ketiga sudah tersedia pada `src/node3` (Kode 9.5) dengan `MY_ADDR 0x0003`; tambahkan `[env:node3]` di `platformio.ini`, lalu set `PEER_ADDR` Node1 menjadi `0xFFFF`. Amati bahwa Node2 dan Node3 menerima frame yang sama, lalu diskusikan apa yang hilang dibanding unicast (tidak ada ACK, tidak ada penyaringan alamat tujuan).

Perhatikan pula **dari alamat mana** PONG di Node1 berasal. Pada uji di perangkat (25 detik, `monitor_serial.py`), Node2 dan Node3 masing-masing menerima 12/12 PING dan membalas semuanya, tetapi seluruh 12 PONG yang dicetak Node1 adalah **`RX dari 0x0003`**. Kedua penerima membalas pada saat yang hampir sama, CCA keduanya melihat channel kosong, dan kedua PONG bertabrakan di udara; Node1 hanya menangkap frame yang lebih kuat (*capture effect*) dan tidak ada ACK yang memberi tahu Node2 bahwa PONG-nya hilang. RSSI menjelaskan siapa yang menang: di Node1, sinyal Node3 sekitar −4 dBm, sedangkan sinyal Node2 sekitar −12 dBm. Tanpa pembacaan `SrcAddr`, kehilangan ini tidak terlihat sama sekali.

```
Link (pengirim -> penerima)      jenis  kirim  terima   loss   RSSI dBm rata2 (min..max)
-------------------------------------------------------------------------------------
Node1(0x0001) -> Node2(0x0002)   PING      12     12    0.0%    -12.0 (-12..-12)
Node1(0x0001) -> Node3(0x0003)   PING      12     12    0.0%     -5.4 (-6..-5)
Node2(0x0002) -> Node1(0x0001)   PONG      12      0  100.0%   -
Node3(0x0003) -> Node1(0x0001)   PONG      12     12    0.0%     -3.8 (-6..-2)
```

Data capture — tabel gejala lintas kelompok

Uji	Konfigurasi	TX Node1?	RX Node2?	Loss	Gejala yang diamati
04-a	PAN sama				
04-b	PAN beda				
04-c	Channel sama				

Uji	Konfigurasi	TX Node1?	RX Node2?	Loss	Gejala yang diamati
04-d	Channel beda				
04-e	Broadcast (3 node)				

Tabel 9.10: Lembar pengamatan EXP-04

CHECKPOINT

Praktikan dapat membedakan 04-b dari 04-d tanpa melihat kode, hanya dari pola log: 04-b ada TX tanpa RX di pasangan; 04-d kedua sisi sunyi. Jika sebuah kelompok di ruangan sama menyalakan radio 802.15.4 di channel yang sama, gejala yang muncul mirip 04-b — itu sebabnya tiap kelompok wajib memakai channel berbeda.

9.8.5 EXP-05 — Promiscuous Mode: Ada dan Tidaknya Filter MAC

EXP-04 menunjukkan bahwa node dengan PAN ID berbeda tidak saling menerima. Penyaringan itu **tidak** terjadi dengan sendirinya: driver ESP-IDF menyalakan promiscuous mode secara default, dan filter PAN ID serta alamat tujuan baru aktif karena `setup()` memanggil satu baris berikut sebelum `esp_ieee802154_set_rx_when_idle(true)`:

```
esp_ieee802154_set_promiscuous(false);  // aktifkan filter PAN ID & alamat di hardware (default driver: true)
```

Percobaan ini membuktikan pengaruh baris tersebut. Seperti EXP-04, yang diubah hanya satu baris di `setup()`; tiap perubahan diikuti unggah ulang node yang bersangkutan. Channel semua node tetap 15.

Uji	PAN (Node1 / Node2)	Baris promiscuous	Gejala yang harus terlihat
05-a	`0xCAFE` / `0xBEEF`	`false` di semua node (kode asli)	Node1 tetap cetak `TX`, Node2 **tidak pernah** cetak `RX` — sama dengan 04-b
05-b	`0xCAFE` / `0xBEEF`	Node2: `true`	Node2 cetak `RX dari 0x0001 (ke 0x0002, PAN 0xCAFE)` untuk setiap PING dan membalas PONG, tetapi Node1 **tidak pernah** cetak `RX`
05-c	`0xCAFE` / `0xBEEF`	Node2: baris **dihapus**	Sama dengan 05-b — tanpa baris ini driver memakai nilai default `true`
05-d	`0xCAFE` / `0xCAFE`	Node3: `true` (Node1, Node2: `false`)	Node3 **ikut** cetak `RX dari 0x0001 (ke 0x0002, ...)` dan membalas; PONG Node2 bertabrakan dengan PONG Node3, sehingga Node1 hanya menerima PONG dari `0x0003` — atau tidak sama sekali

Tabel 9.11: Matriks uji promiscuous mode EXP-05

Ada tiga hal yang harus bisa dijelaskan dari percobaan ini.

Filter bekerja di hardware (05-a vs 05-b) kode callback `esp_ieee802154_receive_done()` memang membaca header, tetapi tidak menyaring berdasarkan PAN ID atau alamat tujuan, sehingga satu-satunya yang membedakan 05-a dan 05-b adalah filter MAC. Begitu promiscuous dinyalakan, PING dari PAN asing langsung sampai ke callback.

Penyaringan terjadi di sisi penerima (05-b) PONG dari Node2 membawa Dest PAN `0xBEEF`, sementara Node1 masih memakai `false` sehingga PONG itu dibuang. Komunikasi hanya "tembus" ke arah node yang promiscuous.

Filter alamat tujuan ikut mati (05-d) promiscuous tidak hanya mengabaikan PAN ID, tetapi juga alamat tujuan. Node3 menerima PING yang ditujukan ke `0x0002` **dan** PONG yang ditujukan ke `0x0001` — keduanya bukan untuknya, padahal Node1 tidak mengirim ke `0xFFFF`. Node3 mencetak `RX dari 0x0001 (ke 0x0002, ...)` lalu ikut membalas PING itu, sehingga PONG-nya bertabrakan dengan PONG Node2. Pada uji 25 detik (`monitor_serial.py`): 12 PING, Node3 menangkap 12/12, dan Node1 hanya menerima 5 PONG — **seluruhnya dari `0x0003`** (RSSI sekitar −3 dBm, jauh di atas sinyal Node2 sekitar −12 dBm); 7 PING lainnya tidak terbalas sama sekali karena kedua PONG rusak. Buffer penerima hanya satu slot, jadi kadang PING di Node3 sudah tertimpa PONG Node2 sebelum sempat dibaca; Node3 lalu mencetak `RX dari 0x0002 (ke 0x0001, ...)`, tidak membalas (bukan PING), dan PONG Node2 lolos ke Node1.

```
Link (pengirim -> penerima)      jenis  kirim  terima   loss   RSSI dBm rata2 (min..max)
-------------------------------------------------------------------------------------
Node1(0x0001) -> Node2(0x0002)   PING      12     12    0.0%    -11.0 (-11..-11)
Node1(0x0001) -> Node3(0x0003)   PING       -     12     n/a     -5.0 (-5..-5)
Node2(0x0002) -> Node1(0x0001)   PONG      12      0  100.0%   -
Node3(0x0003) -> Node1(0x0001)   PONG      12      5   58.3%    -3.2 (-4..-2)
```

Bandingkan dengan XBee: firmware XBee selalu menyaring PAN ID dan alamat tujuan, sehingga perilakunya setara dengan 05-a tanpa perlu diatur. Pada ESP32-H2 yang memakai radio 802.15.4 secara langsung, filter itu harus dinyalakan sendiri.

Catatan. Node2 dan Node3 hanya membalas PING, bukan PONG. Tanpa aturan ini, node yang promiscuous akan membalas PONG milik node lain dan penerimanya membalas balik — terjadi *reply storm* (Serial Monitor dibanjiri puluhan `RX`/`TX` per detik). Dengan aturan ini, menyalakan promiscuous pada Node2 **dan** Node3 bersamaan hanya menghasilkan gejala seperti 04-e: keduanya membalas tiap PING dan Node1 hanya menerima PONG dari salah satunya. Kembalikan semua node ke `false` dan `PAN_ID 0xCAFE` setelah selesai.

Data capture

Uji	Konfigurasi	RX Node1?	RX Node2?	RX Node3?	Gejala yang diamati
05-a	PAN beda, `false`				
05-b	PAN beda, Node2 `true`				
05-c	PAN beda, Node2 tanpa baris				
05-d	PAN sama, Node3 `true`				

Tabel 9.12: Lembar pengamatan EXP-05

CHECKPOINT

Praktikan dapat menjelaskan mengapa 05-b menghasilkan komunikasi **satu arah** (Node2 menerima PING, Node1 tidak menerima PONG), dan mengapa 05-c sama dengan 05-b. Jawaban yang benar menyebut bahwa `esp_ieee802154_set_panid()` hanya mengisi register, sedangkan penyaringan baru aktif bila promiscuous dimatikan. Praktikan juga dapat menjelaskan, dari alamat pengirim dan RSSI di log, mengapa PONG di Node1 pada 05-d berasal dari `0x0003` dan mengapa sebagian PING tidak terbalas sama sekali.

9.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 3 × **ESP32-H2 DevKitM-1** (Node3 menyala tetapi diam), board berdekatan di meja, direkam 17,8 detik dengan `monitor_serial.py`. Baris dengan timestamp sama diurutkan menurut alur protokol.

```
[  0.512] Node1 | Channel 15, PAN 0xCAFE, short addr 0x0001
[  0.512] Node2 | Channel 15, PAN 0xCAFE, short addr 0x0002
[  2.483] Node1 | TX ke 0x0002: PING 1
[  2.483] Node2 | RX dari 0x0001: PING 1  [RSSI -11 dBm, LQI 10]
[  2.483] Node2 | TX balasan ke 0x0001: PONG 1
[  2.483] Node1 | RX dari 0x0002: PONG 1  [RSSI -12 dBm, LQI 11]
[  4.498] Node1 | TX ke 0x0002: PING 2
[  4.498] Node2 | RX dari 0x0001: PING 2  [RSSI -11 dBm, LQI 11]
[  4.498] Node2 | TX balasan ke 0x0001: PONG 2
[  4.498] Node1 | RX dari 0x0002: PONG 2  [RSSI -12 dBm, LQI 11]
```

Parameter	Hasil terukur
PING dikirim / diterima Node2	8 / 8
PONG dikirim / diterima Node1	8 / 8
RSSI PING di Node2 / PONG di Node1	−11 dBm / −12 dBm
LQI (mentah hardware)	9–11
Round-trip PING ke PONG	di bawah resolusi timestamp (PING dan PONG tercetak pada milidetik yang sama)

Tabel 9.13: Hasil verifikasi hardware Modul 07

9.10 Pengukuran

Jarak	RSSI (dBm)	Latency RTT (kasar)	Success PONG (%)
1 m			
3 m			
5 m			
10 m			
15 m			

Tabel 9.14: Lembar pengukuran jarak Modul 07

Pengukuran per-node (pengamatan 2 menit) — pisahkan dua arah.

Node	TX	RX	Loss (%)
Node1 (`0x0001`)			
Node2 (`0x0002`)			

Tabel 9.15: Lembar pengukuran per-node Modul 07

RSSI sudah dicetak di setiap baris RX (disalin dari `frame_info->rssi` di `receive_done`, dicetak di `loop()` — tidak di ISR). Cara termudah mengisi tabel di atas: jalankan `monitor_serial.py --duration 120` pada tiap jarak, lalu ambil RSSI rata-rata (min..max) dan loss per arah dari ringkasannya.

Baseline untuk M16. Catat jarak maksimum yang masih 100 % berhasil pada modul ini. Angka itu adalah jangkauan radio 802.15.4 **tanpa** bantuan mesh — pembanding langsung untuk Zigbee (M10) dan Thread (M12).

9.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Bagaimana pengaruh jarak terhadap RSSI dan persentase PING yang dibalas PONG?
2. Pada jarak berapa komunikasi mulai gagal, dan apa indikasinya di Serial Monitor?
3. Apakah round-trip latency bertambah signifikan dengan jarak? Mengapa?
4. Berapa packet loss tiap arah (PING hilang dibanding PONG hilang)? Adakah asimetri, dan apa dugaan penyebabnya?
5. Mengapa channel dan PAN ID harus sama, dan apa hubungan 802.15.4 dengan Zigbee pada modul berikutnya?

9.12 Concept Check

1. Apa fungsi field Frame Control dan sequence number pada frame 802.15.4?
2. Apa perbedaan short address dan extended address, dan kapan masing-masing dipakai?
3. Mengapa FCS tidak perlu dihitung di perangkat lunak, tetapi panjangnya tetap harus dihitung?
4. Mengapa callback penerimaan tidak boleh mencetak langsung ke Serial (konteks ISR)?
5. Apa yang terjadi bila dua node memakai PAN ID berbeda — di lapisan mana penyaringan itu terjadi?

9.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Sequence number dan ukuran payload

CH-1 — Sequence number nyata (wajib) Field `frame[3]` saat ini selalu `0`. Isi dengan nomor urut yang naik tiap kirim seperti Kode 9.6, lalu hitung packet loss dari sisi penerima. Contoh: 60 PING dikirim, 57 PONG diterima, sehingga loss = (60 − 57)/60 × 100 % = 5 %.

CH-2 — Pengaruh ukuran payload Perbesar payload menjadi 40 byte teks dan bandingkan success rate terhadap payload pendek pada jarak yang sama. Jelaskan hasilnya dari sisi peluang bit error per frame.

```
// buildFrame(): ganti frame[3] = 0;
static uint8_t seq = 0;
frame[3] = seq++;
```

Kode 9.6: Kerangka CH-1 — sequence number pada MHR

Level 3 — Tantangan · RSSI, jarak, dan broadcast

CH-3 — RSSI terhadap jarak dan *capture effect* (a) Dengan `monitor_serial.py --duration 120`, catat RSSI rata-rata (min..max) kedua arah pada kelima jarak di tabel Pengukuran, lalu buat grafik RSSI terhadap jarak. (b) Ulangi uji broadcast 04-e dua kali: sekali dengan Node3 lebih dekat ke Node1 daripada Node2, sekali sebaliknya. Sebelum menjalankan, **prediksi** PONG siapa yang akan diterima Node1 berdasarkan selisih RSSI, lalu buktikan dengan ringkasan monitor. Berapa selisih RSSI minimum agar PONG yang lebih kuat tetap lolos?

CH-4 — Broadcast Ubah `DestAddr` menjadi `0xFFFF` (broadcast) dan tambahkan node ketiga. Amati apakah kedua penerima menerima frame yang sama, dan diskusikan apa yang hilang (tidak ada ACK, tidak ada penyaringan alamat).

9.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (802.15.4, MHR, channel, PAN ID, short address, tiga aturan API).
3. Konfigurasi — environment `node1` dan `node2`, channel 15, PAN `0xCAFE`, interval 2 s.
4. Hasil eksperimen — log PING–PONG kedua node beserta analisis dump frame heksadesimal.
5. Data pengukuran — tabel jarak, tabel per-node, dan baseline jangkauan untuk M16.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-3.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

10. Modul 08 — Zigbee P2P: Join dan Binding

IoT COMMUNICATION LAB · MODUL 08

Join a Zigbee Network — Zigbee P2P: Join dan Binding

ESP32-H2 · Zigbee · ZC ke ZED · level Advanced · 3 × 50 menit · folder kode `week08_zigbee_p2p`

PENGANTAR MODUL · Gambaran Umum

Modul 08 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat lanjut. Misinya membentuk jaringan Zigbee, memasukkan satu end device ke dalamnya, dan mengendalikannya lewat binding. Percobaan berjalan sebagai Zigbee P2P antara coordinator yang berperan sebagai switch dan end device yang berperan sebagai lampu, diamati melalui dua terminal Serial Monitor pada 115200 baud dengan LED RGB bawaan sebagai penanda visual.

10.1 Pendahuluan

Pada M07 frame disusun sendiri: tidak ada identitas jaringan, tidak ada keamanan, tidak ada penemuan perangkat. Zigbee menambahkan ketiganya di atas radio 802.15.4 yang **sama persis**. Yang menarik justru harga yang harus dibayar: firmware jauh lebih besar (butuh tabel partisi khusus), dan perangkat menyimpan keanggotaan jaringan di NVS — sesuatu yang tidak pernah jadi masalah di modul-modul BLE.

Prasyaratnya adalah M07: channel, PAN ID, dan pemahaman bahwa Zigbee berjalan di radio 802.15.4 yang sama. Yang dibangun di sini adalah pembentukan PAN, window join, prosedur find-and-bind, cluster ON/OFF, perbedaan mode build ZCZR dan ED, tabel partisi khusus Zigbee, serta penyimpanan keanggotaan jaringan di NVS. Semuanya dipakai lagi pada M09 ketika satu coordinator melayani banyak end device melalui binding table, M10 ketika router menambah hop, dan M16 ketika Zigbee menjadi salah satu protokol yang dibandingkan.

Peta modul blok Zigbee

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
07	802.15.4 telanjang — frame disusun manual
08 (ini)	**Jaringan Zigbee terbentuk: join, binding, cluster ON/OFF**
09	Satu coordinator melayani banyak end device (binding table)
10	Router menambah hop — routing multi-hop otomatis

Tabel 10.1: Peta modul blok Zigbee

Kontrak data lab ini. Zigbee tidak mengirim "string", melainkan **perintah cluster standar** (`On/Off`). Perbedaan ini penting untuk M16: BLE dan Thread di lab ini mengirim payload bebas, Zigbee mengirim perintah baku. Catat konsekuensinya pada interoperabilitas — perangkat Zigbee merek berbeda bisa saling mengerti, payload BLE buatan sendiri tidak.

10.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membentuk jaringan Zigbee dan mengendalikan lewat binding

1. Membentuk jaringan Zigbee dengan satu coordinator dan memasukkan satu end device melalui window join 180 detik, dibuktikan dari log kedua board.
2. Menjelaskan perbedaan **join** dan **binding** dengan menunjuk baris log yang menandai masing-masing tahap.
3. Mengukur waktu join dan binding serta latency perintah ON/OFF (coordinator sampai LED berubah di end device) pada minimal 4 jarak.
4. Menjelaskan mengapa dua peran membutuhkan `build_flags` dan tabel partisi berbeda, serta akibatnya bila tertukar.

Kriteria keberhasilan

- ☐ Coordinator mencetak `End device ter-binding!`.
- ☐ LED RGB pada end device berubah ON/OFF tiap 5 detik mengikuti perintah.
- ☐ Waktu join dan binding tercatat, minimal 3 percobaan.
- ☐ Tabel jarak–latency–success terisi dari pengukuran sendiri.

10.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (ZDO, APS layer, key management, Zigbee Cluster Library lengkap) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 10.2.

Istilah	Definisi kerja di lab ini
Zigbee	Protokol jaringan di atas IEEE 802.15.4 untuk otomasi; dipakai lewat library `Zigbee` Arduino core 3.x.
Coordinator (ZC)	Membentuk dan mengelola PAN; build flag `-DZIGBEE_MODE_ZCZR`.
End Device (ZED)	Node akhir hemat daya, tidak meneruskan trafik; build flag `-DZIGBEE_MODE_ED`.
Endpoint	"Nomor kamar" fungsi di dalam satu perangkat: switch EP 5, light EP 10.
Join	End device masuk ke jaringan yang dibuka coordinator (`setRebootOpenNetwork(180)`).
Binding	Mengaitkan endpoint switch dengan endpoint light (find-and-bind) agar perintah punya tujuan.
Cluster ON/OFF	Perintah standar Zigbee untuk menyalakan atau mematikan beban (`lightOn()` dan `lightOff()`).
Partisi khusus	`partitions_zczr.csv` dan `partitions_ed.csv` menyediakan `zb_storage` dan `zb_fct`.
NVS jaringan	Keanggotaan Zigbee disimpan di flash — bertahan melewati reset **dan** melewati flash ulang firmware.

Tabel 10.2: Istilah kerja Modul 08

Join berbeda dari binding. Join membuat perangkat menjadi **anggota jaringan** (punya alamat, punya kunci). Binding membuat satu endpoint **tahu harus mengirim ke endpoint mana**. Perangkat bisa sudah join tetapi belum ter-binding — dan perintahnya tidak akan sampai ke mana pun. Dua tahap ini muncul sebagai dua baris log yang berbeda; pastikan keduanya dapat ditunjuk.

Dari XBee ke ESP32-H2: di mana MY, SH, dan SL? Kalau sebelumnya Anda belajar komunikasi dengan modul XBee, semua konsep alamat di sana tetap berlaku — hanya cara mengaksesnya yang berbeda. Di XBee, parameter dibaca dan diatur dengan perintah AT (`ATMY`, `ATSH`, ...) lewat XCTU atau terminal. Di ESP32-H2, nilai yang sama dibaca dari stack ESP-Zigbee oleh kode, lalu dicetak fungsi `printNetworkInfo()`. Padanan lengkapnya ada pada Tabel 10.3.

```
Short address  : 0x7EC7                   ← MY  (alamat logika 16-bit)
IEEE address   : 74:4D:BD:FF:FE:61:E8:C1  ← SH + SL (alamat fisik 64-bit)
                 └── SH ──┘ └── SL ──┘
                 0x744DBDFF  0xFE61E8C1
```

XBee (AT)	Arti	Di ESP32-H2 / log modul ini	Catatan
MY	Alamat **logika** 16-bit (network address)	`Short address` — `esp_zb_get_short_address()`	Di Zigbee **dibagikan coordinator saat join**, hanya bisa dibaca; coordinator selalu `0x0000`. Bisa berubah bila node join ulang.
SH	32 bit **atas** alamat **fisik** 64-bit	4 byte pertama `IEEE address` — `esp_zb_get_long_address()`	Ditanam pabrik, tetap, unik sedunia — sama seperti nomor di label XBee.
SL	32 bit **bawah** alamat fisik 64-bit	4 byte terakhir `IEEE address`	
CH	Channel operasi	`Channel` — `esp_zb_get_current_channel()`	XBee menulisnya heksadesimal: channel 18 = `0x12`.
SC	Daftar channel yang boleh di-scan	`Zigbee.setPrimaryChannelMask()`	Default semua channel 11–26.
ID	Extended PAN ID yang **diminta**	Tidak diatur di kode modul ini	ID = 0 di XBee berarti "coordinator yang memilih"; di sini coordinator memakai IEEE address-nya sendiri.
OP	Extended PAN ID 64-bit yang **dipakai**	`Extended PAN ID` — `esp_zb_get_extended_pan_id()`	
OI	PAN ID 16-bit yang **dipakai**	`PAN ID` — `esp_zb_get_pan_id()`	Dipilih acak oleh coordinator saat membentuk network.
CE	Coordinator Enable (peran node)	Build flag `ZIGBEE_MODE_ZCZR` / `ZIGBEE_MODE_ED` + `Zigbee.begin(ZIGBEE_COORDINATOR)`	Di ESP32 peran dipilih saat **build**, bukan lewat perintah AT.
NJ	Lama network terbuka untuk join	`Zigbee.setRebootOpenNetwork(180)`	Sama-sama dalam detik.
AI	Status asosiasi (0 = sudah join)	`Zigbee.connected()`	
DH / DL	Alamat 64-bit **tujuan** (mode transparan)	**Tidak ada padanan langsung** — tujuan ditentukan **binding table**	Lihat penjelasan di bawah.
Endpoint `0xE8` (DE/SE)	Endpoint data transparan Digi	Endpoint 5 (switch) dan 10 (light)	Zigbee standar memakai endpoint + cluster (On/Off = `0x0006`), bukan satu endpoint data umum.

Tabel 10.3: Padanan parameter XBee dan ESP32-H2 pada Modul 08

Tiga perbedaan yang paling sering membingungkan:

1. **MY tidak Anda isi sendiri.** Pada XBee 802.15.4 (Series 1) dan pada M07, alamat 16-bit ditentukan pengguna (`0x0001`, `0x0002`). Pada Zigbee — baik XBee Zigbee (Series 2/3) maupun ESP32-H2 — alamat

16-bit **diberikan coordinator** saat join. Karena bisa berubah, alamat yang pasti untuk mengenali satu perangkat adalah **SH+SL (IEEE address)**, bukan MY.

2. **Tidak ada DH/DL.** Di XBee mode transparan, Anda menulis alamat tujuan ke DH/DL lalu semua data serial terkirim ke sana. Di Zigbee standar, switch mengirim *command* `On`/`Off` ke **endpoint** yang tercatat di **binding table** — isinya diisi otomatis oleh find-and-bind. Karena itu log coordinator mencetak `End device ter-binding!`, bukan alamat tujuan.
3. **Yang dikirim bukan byte mentah, melainkan command cluster.** XBee transparan meneruskan byte apa pun yang masuk ke UART-nya. `lightOn()` mengirim command ZCL `On` pada cluster On/Off, dan penerimanya harus endpoint yang memang mengerti cluster itu (`ZigbeeLight`).

`ZigbeeSwitch` dan `ZigbeeLight` bukan fitur standar Zigbee. Keduanya class C++ buatan Espressif di library `Zigbee` Arduino core 3.x, yang membungkus *device type* standar (On/Off Light Switch dan On/Off Light) beserta cluster On/Off, endpoint, dan find-and-bind. Yang standar adalah cluster, device type, dan command-nya (`On`, `Off`, `Toggle`); nama method seperti `bound()`, `allowMultipleBinding()`, dan `onLightStateChange()` adalah API Espressif. Pembagian perannya dirangkum pada Tabel 10.4.

Class	Mewakili	Yang disembunyikan class ini
`ZigbeeSwitch`	On/Off Light Switch (cluster On/Off sisi *client*, mengirim command)	Membuat endpoint, mendaftarkan cluster, find-and-bind. `lightOn()` mengirim command ZCL `On` ke tujuan di binding table.
`ZigbeeLight`	On/Off Light (cluster On/Off sisi *server*, menerima command)	Menerima command lalu memanggil callback `onLightChange`.

Tabel 10.4: Class dasar yang dipakai pada Modul 08

Class lain di library `Zigbee`. Semua class endpoint mewarisi `ZigbeeEP` dan dipakai dengan pola yang sama: buat objek dengan nomor endpoint, `setManufacturerAndModel()`, `Zigbee.addEndpoint()`, lalu `Zigbee.begin()`. Objek global `Zigbee` mengelola stack (`begin()`, `connected()`, `setRebootOpenNetwork()`). Daftar class dikelompokkan pada Tabel 10.5.

Kategori	Class
Lampu dan saklar	`ZigbeeLight`, `ZigbeeSwitch`, `ZigbeeColorDimmableLight`, `ZigbeeColorDimmerSwitch`
Sensor	`ZigbeeTempSensor`, `ZigbeeFlowSensor`, `ZigbeePressureSensor`, `ZigbeeOccupancySensor`, `ZigbeeContactSwitch`, `ZigbeeVibrationSensor`, `ZigbeeIlluminanceSensor`, `ZigbeeCarbonDioxideSensor`
Aktuator	`ZigbeePowerOutlet`, `ZigbeeWindowCovering`, `ZigbeeFanControl`, `ZigbeeDoorWindowHandle`
Kontrol dan ukur	`ZigbeeThermostat`, `ZigbeeElectricalMeasurement`
Generik	`ZigbeeAnalog`, `ZigbeeBinary`, `ZigbeeGateway`

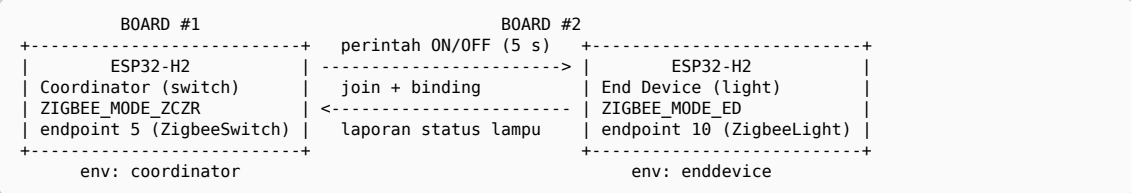
Tabel 10.5: Class endpoint pada library `Zigbee`

> **CATATAN**
> Daftar ini bergantung pada versi Arduino core; sumber pastinya ada di `libraries/Zigbee/src/ep/` pada repositori `espressif/arduino-esp32`. `ZigbeeTempSensor` paling relevan untuk WSN karena mengirim data sensor lewat attribute reporting, bukan hanya perintah On/Off.

Sekuens protokol yang diamati

```
Coordinator boot ⟶ Zigbee.begin(ZIGBEE_COORDINATOR)
   ⟶ network terbuka 180 s ⟶ ED join ⟶ find-and-bind switch↔light
   ⟶ loop 5 s: lightOn() / lightOff() ⟶ ED: setLED ON/OFF
```

10.4 Topologi



Node	Board	Environment	Peran	Endpoint
Coordinator	ESP32-H2 DevKitM-1	`coordinator`	ZC dan switch	EP 5
End device	ESP32-H2 DevKitM-1	`enddevice`	ZED dan light	EP 10

Tabel 10.6: Peran tiap node Modul 08

Kedua peran memakai **ESP32-H2 DevKitM-1**. Perbedaannya hanya pada `build_flags` (`-DZIGBEE_MODE_ZCZR` dibanding `-DZIGBEE_MODE_ED`) dan tabel partisi — board-nya identik, jadi board mana pun bisa diberi peran mana pun.

10.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan library `Zigbee` bawaan.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1, **dengan LED RGB bawaan** (`RGB_BUILTIN`)	2
2	Kabel USB data	kabel data, bukan *charge-only*	2
3	PC/Laptop	PlatformIO Core/IDE, 2 port USB bebas	1
4	Platform PlatformIO	pioarduino `espressif32` 55.03.311 (library `Zigbee` bawaan core 3.x)	—
5	Tabel partisi	`partitions_zczr.csv` (ZC) dan `partitions_ed.csv` (ED) — sudah ada di folder modul	—

Tabel 10.7: Alat dan bahan Modul 08

10.6 Kode Program

Dua peran, dua tabel partisi. Potongan `platformio.ini` pada Kode 10.1 memperlihatkan perbedaan `build_flags` dan `board_build.partitions` antara kedua environment.

```
[env:coordinator]
build_src_filter = +<coordinator/*.cpp>
build_flags =
    -DZIGBEE_MODE_ZCZR
    -lesp_zb_api.zczr
    -lzboss_stack.zczr
    -lzboss_port.native
board_build.partitions = partitions_zczr.csv
upload_port  = /dev/ttyACM0
monitor_port = /dev/ttyACM0

[env:enddevice]
build_src_filter = +<enddevice/*.cpp>
build_flags =
    -DZIGBEE_MODE_ED
    -lesp_zb_api.ed
    -lzboss_stack.ed
    -lzboss_port.native
board_build.partitions = partitions_ed.csv
upload_port  = /dev/ttyACM2
monitor_port = /dev/ttyACM2
```

Kode 10.1: Potongan `platformio.ini` — dua peran dengan tabel partisi berbeda

```
[platformio]
default_envs = coordinator

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200

[env:coordinator]
build_src_filter = +<coordinator/*.cpp>
build_flags =
    -DZIGBEE_MODE_ZCZR
    -lesp_zb_api.zczr
    -lzboss_stack.zczr
    -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:enddevice]
build_src_filter = +<enddevice/*.cpp>
build_flags =
    -DZIGBEE_MODE_ED
    -lesp_zb_api.ed
    -lzboss_stack.ed
    -lzboss_port.native
board_build.partitions = partitions_ed.csv
```

Kode 10.2: `platformio.ini` Modul 08 pada repositori

```
# Name,     Type, SubType, Offset,  Size, Flags
nvs,        data, nvs,     0x9000,  0x5000,
otadata,    data, ota,     0xe000,  0x2000,
app0,       app,  ota_0,   0x10000, 0x140000,
app1,       app,  ota_1,   0x150000,0x140000,
spiffs,     data, spiffs,  0x290000,0x15A000,
zb_storage, data, fat,     0x3EA000,0x4000,
zb_fct,     data, fat,     0x3EE000,0x1000,
rcp_fw,     data, spiffs,  0x3EF000,0x1000,
coredump,   data, coredump,0x3F0000,0x10000,
```

Kode 10.3: `partitions_zczr.csv` — tabel partisi peran coordinator/router

```
# Name,     Type, SubType, Offset,  Size, Flags
nvs,        data, nvs,     0x9000,  0x5000,
otadata,    data, ota,     0xe000,  0x2000,
app0,       app,  ota_0,   0x10000, 0x140000,
app1,       app,  ota_1,   0x150000,0x140000,
spiffs,     data, spiffs,  0x290000,0x15B000,
zb_storage, data, fat,     0x3EB000,0x4000,
zb_fct,     data, fat,     0x3EF000,0x1000,
coredump,   data, coredump,0x3F0000,0x10000,
```

Kode 10.4: `partitions_ed.csv` — tabel partisi peran end device

CATATAN
Kedua berkas partisi di atas identik isinya pada modul M08, M09, dan M10, sehingga listing yang sama tidak diulang pada bab-bab berikutnya.

```ino
// Minggu 8 — Zigbee P2P: Coordinator (switch) mengendalikan End Device (lampu)
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Mode Zigbee ZCZR belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

ZigbeeSwitch zbSwitch = ZigbeeSwitch(5);  // endpoint 5

void onLightStateChange(bool state) {
  Serial.printf("Lampu sekarang: %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang sedang dipakai. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Network Zigbee terbentuk:");
  Serial.println("  Peran          : Coordinator (ZC)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 5 (switch)");
}

void setup() {
  Serial.begin(115200);

  zbSwitch.setManufacturerAndModel("Espressif", "ZigbeeSwitch");
  zbSwitch.allowMultipleBinding(false);   // P2P: hanya satu light
  zbSwitch.onLightStateChange(onLightStateChange);

  Zigbee.addEndpoint(&zbSwitch);

  // Buka network 180 detik agar end device bisa join
  Zigbee.setRebootOpenNetwork(180);
  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin(ZIGBEE_COORDINATOR)) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  // Bagi coordinator, begin() baru kembali setelah network terbentuk (boot
  // pertama) atau dipulihkan dari NVS (reboot). JANGAN menunggu
  // Zigbee.connected() di sini: pada reboot dengan setRebootOpenNetwork(),
  // library Arduino core 3.3.x tidak pernah men-set flag itu untuk coordinator.
  printNetworkInfo();
```

```
  Serial.println("Menunggu end device ter-binding...");
  while (!zbSwitch.bound()) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nEnd device ter-binding!");
}

void loop() {
  static unsigned long last = 0;
  static bool on = false;
  if (millis() - last > 5000) {
    last = millis();
    on = !on;
    if (on) {
      zbSwitch.lightOn();
      Serial.println("Perintah: Lampu ON");
    } else {
      zbSwitch.lightOff();
      Serial.println("Perintah: Lampu OFF");
    }
  }
}
```

Kode 10.5: `src/coordinator/main.cpp` — coordinator sekaligus switch (EP 5)

```
// Minggu 8 — Zigbee P2P: End Device (lampu) dikendalikan Coordinator
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Mode Zigbee ED belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

uint8_t led = RGB_BUILTIN;

ZigbeeLight zbLight = ZigbeeLight(10);  // endpoint 10

void setLED(bool state) {
  digitalWrite(led, state);
  Serial.printf("Lampu %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang didapat saat join. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Info network:");
  Serial.println("  Peran          : End Device (ED)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X (diberikan coordinator)\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 10 (light)");
}

void setup() {
  Serial.begin(115200);

  pinMode(led, OUTPUT);
  digitalWrite(led, LOW);

  zbLight.setManufacturerAndModel("Espressif", "ZBLightBulb");
  zbLight.onLightChange(setLED);

  Zigbee.addEndpoint(&zbLight);

  Zigbee.setPrimaryChannelMask(1UL << 15);
```

```
  if (!Zigbee.begin()) {  // default = ZIGBEE_END_DEVICE
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  Serial.println("Menunggu bergabung ke network koordinator...");
  while (!Zigbee.connected()) {
    delay(100);
  }
  Serial.println("Berhasil bergabung ke network!");
  printNetworkInfo();
}

void loop() {
  delay(100);
}
```

Kode 10.6: `src/enddevice/main.cpp` — end device sekaligus light (EP 10)

10.6.1 Channel antar Kelompok (wajib kelas paralel)

Satu ruangan praktikum biasanya menampung beberapa kelompok, sehingga berdiri beberapa jaringan Zigbee sekaligus. Satu node tetap hanya menjadi anggota satu jaringan; yang harus dicegah adalah dua hal: node yang baru di-`erase` dapat **salah join** ke coordinator kelompok lain bila beberapa window join terbuka bersamaan, dan beberapa jaringan pada channel yang sama saling berebut airtime sehingga angka loss serta latency — data pengukuran modul ini — tercemar oleh tetangga.

Channel operasi ditentukan coordinator saat membentuk jaringan (baris **SC** pada Tabel 10.3), jadi nilainya disetel lewat `setPrimaryChannelMask()` sebelum `Zigbee.begin()` pada **kedua** berkas (Kode 10.5 dan Kode 10.6). Jatah untuk enam kelompok ada pada Tabel 10.8 — contoh pada listing memakai jatah Kelompok 1.

Kelompok	Channel	Baris yang disisipkan sebelum `Zigbee.begin(...)`
1	15	`Zigbee.setPrimaryChannelMask(1UL << 15);`
2	20	`Zigbee.setPrimaryChannelMask(1UL << 20);`
3	25	`Zigbee.setPrimaryChannelMask(1UL << 25);`
4	11	`Zigbee.setPrimaryChannelMask(1UL << 11);`
5	17	`Zigbee.setPrimaryChannelMask(1UL << 17);`
6	22	`Zigbee.setPrimaryChannelMask(1UL << 22);`

Tabel 10.8: Jatah channel untuk enam kelompok — setiap kelompok memakai mask-nya sendiri

Mask Zigbee adalah satu bit per channel (`1UL << n` berarti **hanya** channel n); mask default `0x07FFF800` mencakup channel 11–26 sehingga coordinator bebas memilih di antaranya.

> **PERINGATAN**
> Pada coordinator baris ini menentukan channel jaringan yang dibentuk. Pada `src/enddevice/main.cpp` baris yang sama **bukan** memilih channel — end device tetap mengikuti channel coordinator — melainkan membatasi channel yang boleh dipindai, supaya hanya jaringan kelompok sendiri yang dapat ditemukan.

> **TIP**
> Channel 15, 20, dan 25 adalah pilihan klasik Zigbee karena paling longgar terhadap Wi-Fi 2,4 GHz; 11, 17, dan 22 melengkapi sebaran agar antar kelompok terpisah minimal dua kanal. Mengganti channel berarti mengganti jaringan yang tersimpan di NVS, jadi **jalankan `-t erase` di kedua board** sebelum flash ulang. Setelah join, cocokkan `Channel` dan `Extended PAN ID` pada kedua log: `Channel` harus sesuai jatah kelompok dan `Extended PAN ID` harus IEEE address coordinator sendiri. Bila board menampilkan channel kelompok lain, NVS-nya masih menyimpan keanggotaan lama — erase lalu join ulang.

10.7 Build dan Flash

Coordinator dulu, lalu end device **dalam 180 detik**.

Linux/macOS (bash):

```
pio run -d week08_zigbee_p2p -e enddevice   -t erase     # bersihkan NVS
pio run -d week08_zigbee_p2p -e coordinator -t erase
pio run -d week08_zigbee_p2p -e coordinator -t upload -t monitor
pio run -d week08_zigbee_p2p -e enddevice   -t upload    # jangan lewat 180 s
```

Windows (PowerShell):

```
$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'   # sekali per sesi terminal
pio run -e coordinator -t erase  --upload-port COM5    # port contoh; ganti
pio run -e enddevice   -t erase  --upload-port COM11   # sesuai pio device list
pio run -e coordinator -t upload --upload-port COM5
pio run -e enddevice   -t upload --upload-port COM11   # dalam 180 s
```

> **CATATAN**
> Prefix `$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'` mencegah `pio run` **menggantung tanpa batas** saat upload/erase di Windows — progress bar esptool memakai karakter Unicode (▉, ▒) yang tidak terwakili codepage default Windows. Setel ulang setiap membuka terminal baru. Erase cukup dijalankan sekali sebelum flash pertama, atau saat berganti channel/kelompok; mengulang percobaan dengan firmware yang sama tidak perlu erase lagi.

> **PENTING**
> **Pilih port USB-to-UART, bukan USB native.** Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- ☐ `pio device list` dijalankan, dua port dicatat dan diisikan di atas.
- ☐ Channel kelompok disetel pada kedua berkas dan berbeda dari kelompok lain — lihat Tabel 10.8.
- ☐ **Hapus NVS kedua board sebelum flash pertama:** `pio run -d week08_zigbee_p2p -e enddevice -t erase`. Tanpa ini, board yang pernah join jaringan lain akan terus mencari koordinator lama.
- ☐ Environment `coordinator` (ZCZR) dan `enddevice` (ED) dikenali PlatformIO.
- ☐ Serial Monitor 115200 baud untuk kedua board.
- ☐ Endpoint dipahami: switch EP 5, light EP 10.
- ☐ Tidak ada jaringan Zigbee lain aktif di sekitar (hindari interferensi dan salah join).

10.8 Percobaan

10.8.1 EXP-01 — Pembentukan Jaringan (Coordinator)

Unggah environment `coordinator`, buka Serial Monitor, verifikasi: coordinator membentuk network, mencetak parameternya, membuka network 180 detik, lalu menunggu binding.

```
boot ──► begin(ZIGBEE_COORDINATOR) OK
     ──► network terbentuk: cetak channel, PAN ID, alamat
     ──► network terbuka 180 s
     ──► menunggu: . . . . . (titik tiap 500 ms)
```

Channel dan PAN ID **tidak ditulis di kode**: coordinator memilih sendiri saat membentuk network (channel dari hasil scan, PAN ID acak), lalu menyimpannya di NVS. Nilainya dibaca dari stack ESP-Zigbee (`esp_zb_get_current_channel()`, `esp_zb_get_pan_id()`, dan seterusnya) di fungsi `printNetworkInfo()`, karena library Arduino hanya menuliskannya ke log debug.

Data capture

Parameter	Hasil
Pesan awal coordinator	
Channel	
PAN ID / Extended PAN ID	
Short address coordinator	
Lama network terbuka (s)	
Endpoint switch	
Perintah yang dikirim tiap 5 s	

Tabel 10.9: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA
Bandingkan ukuran firmware modul ini dengan modul BLE (lihat ringkasan `RAM/Flash` di akhir `pio run`). Lalu buka `partitions_zczr.csv` (Kode 10.3) dan temukan partisi `zb_storage` dan `zb_fct`. Jawab: apa yang disimpan di sana, dan mengapa tabel partisi default Arduino tidak cukup?

CHECKPOINT

Coordinator mencetak titik-titik `.` berulang. Jika langsung muncul `Zigbee gagal start!` lalu board restart, tabel partisi atau `build_flags` tidak cocok — perbaiki sebelum lanjut.

10.8.2 EXP-02 — Join dan Binding (End Device)

Unggah environment `enddevice` ke board kedua **sebelum 180 detik habis**. Amati proses join lalu binding, dan verifikasi RGB bawaan berkedip ON/OFF tiap 5 detik.

```
ED boot ──► Zigbee.begin() ──► scan channel
        ──► join network ──► find & bind ke switch
ZC loop 5 s: lightOn() … lightOff() … (bergantian)
```

Expected output — Coordinator (nilai channel, PAN ID, dan alamat dari uji di board; di board Anda akan berbeda)

```
Network Zigbee terbentuk:
  Peran          : Coordinator (ZC)
  Channel        : 18
  PAN ID         : 0x4FC6
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x0000
  IEEE address   : 74:4D:BD:FF:FE:61:E6:2C
  Endpoint       : 5 (switch)
Menunggu end device ter-binding...
End device ter-binding!
Perintah: Lampu ON
Perintah: Lampu OFF
Perintah: Lampu ON
Perintah: Lampu OFF
```

Bila binding berlangsung lambat, baris titik-titik (`....`) tercetak di bawah `Menunggu end device ter-binding...` selama menunggu.

> **CATATAN**
> Baris `Lampu sekarang: ...` berasal dari callback `onLightStateChange()`, yaitu **laporan balik** dari lampu ke switch. Pada firmware Arduino core 3.3.x baris ini tidak muncul karena `ZigbeeLight` tidak mengonfigurasi attribute reporting ke switch secara otomatis — perintah tetap sampai dan lampu tetap menyala. Mengaktifkan laporan balik justru menjadi CH-2.

Expected output — End device

```
Menunggu bergabung ke network koordinator...
Berhasil bergabung ke network!
Info network:
  Peran          : End Device (ED)
  Channel        : 18
  PAN ID         : 0x4FC6
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x7EC7 (diberikan coordinator)
  IEEE address   : 74:4D:BD:FF:FE:61:E8:C1
  Endpoint       : 10 (light)
Lampu ON
Lampu OFF
```

Padanan XBee (Tabel 10.3): `Short address` = **MY**, `IEEE address` = **SH + SL**, `Channel` = **CH**, `PAN ID` = **OI**, `Extended PAN ID` = **OP**.

Bandingkan info network kedua board: **Channel, PAN ID, dan Extended PAN ID harus sama** — itulah bukti end device bergabung ke network coordinator ini, bukan network lain. Short address coordinator selalu `0x0000`; short address end device dibagikan coordinator saat join. Bila tidak diatur di kode, Extended PAN ID diambil dari IEEE address coordinator — terlihat pada log di atas.

> **CHECKPOINT**
> Dua hal harus terjadi berurutan: end device mencetak `Berhasil bergabung ke network!` (join), lalu coordinator mencetak `End device ter-binding!` (binding). Jika join berhasil tetapi binding tidak pernah terjadi, perintah tidak akan sampai — jangan lanjut, ulangi dengan menghapus NVS kedua board.

10.8.3 EXP-03 — Keandalan Kontrol

1. Hitung siklus ON/OFF per menit (harapan 12) dan bandingkan status di kedua Serial Monitor.
2. Reset end device — catat waktu join ulang dan verifikasi apakah keanggotaan tersimpan (network terbuka lagi tiap coordinator reboot 180 s).
3. **Cabut daya coordinator selama 30 detik**, lalu nyalakan lagi. Amati apakah end device otomatis kembali atau perlu di-reset.
4. Geser jarak bertahap dan catat kapan perintah mulai tertinggal.

Data capture

Parameter	Hasil
Siklus ON/OFF per menit	
Waktu join ulang setelah reset ED	
Perilaku saat coordinator mati lalu hidup	
Sinkron status ZC dan ED?	
Jarak mulai ada perintah tertinggal (m)	

Tabel 10.10: Lembar pengamatan EXP-03

CHECKPOINT

Praktikan dapat menjelaskan mengapa end device yang di-reset bisa kembali **tanpa** window join dibuka lagi (petunjuk: keanggotaan tersimpan di NVS, join hanya diperlukan sekali).

10.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 2 × **ESP32-H2 DevKitM-1** (flash di-erase lebih dulu agar tidak ada sisa jaringan Zigbee lama di NVS), capture 70 detik.

```
# Coordinator (ESP32-H2, ZCZR)          # End device (ESP32-H2, ED)
[0.401] Menunggu end device ...         [0.401] Menunggu bergabung ...
[3.407] ......                          [3.205] Berhasil bergabung ke network!
[3.407] End device ter-binding!
[5.209] Perintah: Lampu ON              [5.207] Lampu ON
[10.219] Perintah: Lampu OFF            [10.214] Lampu OFF
```

Parameter	Hasil terukur
Waktu join dan binding sejak boot	3,4 s
Perintah dikirim coordinator	13
Aksi terjadi di end device	13 (0 % loss)
Selisih waktu perintah sampai aksi	≈ 2 ms
Siklus ON/OFF per menit	12

Tabel 10.11: Hasil verifikasi hardware Modul 08

10.10 Pengukuran

Jarak	RSSI (dBm)	Latency perintah (kasar)	Success ON/OFF (%)
1 m			
3 m			
5 m			
10 m			
15 m			

Tabel 10.12: Lembar pengukuran jarak Modul 08

Pengukuran per-node (pengamatan 2 menit, jarak tetap).

Node	Perintah terkirim	Aksi terlaksana	Loss (%)
Coordinator (switch)		—	
End device (light)	—		

Tabel 10.13: Lembar pengukuran per-node Modul 08

Waktu join dan binding (minimal 3 percobaan, hapus NVS tiap kali).

Percobaan	Waktu join (s)	Waktu binding (s)	Total (s)
1			
2			
3			

Tabel 10.14: Lembar pengukuran waktu join dan binding

Bandingkan dengan M07. Pada jarak yang sama, apakah success rate Zigbee lebih baik, sama, atau lebih buruk daripada raw 802.15.4? Radionya identik, jadi selisih apa pun berasal dari lapisan di atasnya.

10.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Bagaimana pengaruh jarak terhadap keberhasilan perintah ON/OFF diterima lampu?
2. Apakah latency perintah (ZC hingga LED berubah di ED) bertambah pada jarak jauh?
3. Berapa persen perintah gagal dalam 2 menit pengamatan, dan pola gagalnya bagaimana (acak atau berkelompok)?
4. Berapa waktu proses join dan binding dari tiga percobaan, dan apa yang membuatnya bervariasi?
5. Apakah Zigbee (join dan binding otomatis ditambah enkripsi) lebih cocok untuk WSN dibanding raw 802.15.4 M07? Sebutkan apa yang diperoleh dan apa harga yang dibayar.

10.12 Concept Check

1. Apa fungsi coordinator dalam jaringan Zigbee?
2. Apa perbedaan proses join dan binding? Bisakah salah satu terjadi tanpa yang lain?
3. Mengapa end device memakai partisi dan library berbeda (`partitions_ed.csv`, `esp_zb_api.ed`)?
4. Apa yang terjadi bila end device dinyalakan setelah window 180 detik berakhir, dan bagaimana cara memperbaikinya?
5. Mengapa `allowMultipleBinding(false)` dipakai pada skenario P2P ini, dan apa yang berubah di Modul 09?

10.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Mengukur latency dan menutup arah balik

CH-1 — Latency terukur Ukur latency perintah secara objektif: kirim timestamp di payload atau catat `millis()` saat `lightOn()` di coordinator dan saat `setLED()` di end device, lalu bandingkan. Lakukan 20 kali dan hitung rata-rata serta sebarannya.

CH-2 — Laporan balik (menutup celah `onLightStateChange`) Aktifkan attribute reporting pada `ZigbeeLight` sehingga coordinator benar-benar mencetak `Lampu sekarang: ON/OFF`. Bandingkan jumlah perintah terkirim dengan laporan diterima untuk menghitung loss arah balik.

Level 3 — Tantangan · Kontrol dua arah dan ketahanan NVS

CH-3 — Kontrol dari end device Tambahkan tombol pada end device yang mengirim perintah toggle ke coordinator. Diskusikan mengapa ini memerlukan binding arah sebaliknya.

CH-4 — Uji ketahanan NVS Flash ulang end device **tanpa** `-t erase`, lalu amati apakah ia langsung kembali ke jaringan. Jelaskan perbedaannya dengan perangkat BLE di M01–M06 yang tidak menyimpan apa pun.

10.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (Zigbee, ZC dan ZED, endpoint, join dibanding binding, cluster ON/OFF).
3. Konfigurasi — `build_flags`, tabel partisi, endpoint, window join 180 s.
4. Hasil eksperimen — log kedua board (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — tabel jarak, tabel per-node, tabel waktu join dan binding, serta perbandingan dengan M07.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-2.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

11. Modul 09 — Zigbee Multi-Node dan Binding Table

IoT COMMUNICATION LAB · MODUL 09

Scale the Network — Zigbee Multi-Node dan Binding Table

ESP32-H2 · Zigbee · 1 ZC + 2 ZED · level Intermediate · 3 × 50 menit · folder kode `week09_zigbee_multinode`

PENGANTAR MODUL · Gambaran Umum

Modul 09 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat menengah. Misinya menskalakan satu coordinator ke banyak end device tanpa satu perintah pun hilang. Percobaan berjalan sebagai jaringan Zigbee multi-node dengan satu coordinator dan dua end device, diamati melalui tiga terminal Serial Monitor pada 115200 baud dengan LED RGB bawaan sebagai penanda visual.

11.1 Pendahuluan

M08 mengikat **satu** lampu ke satu switch. Di sini `allowMultipleBinding(true)` dinyalakan, dan coordinator harus menyimpan **daftar** tujuan — inilah binding table. Masalah yang muncul identik dengan M05 di dunia BLE (satu pusat, banyak sumber), tetapi jawabannya berbeda: BLE memakai objek koneksi per node, Zigbee memakai `endpoint + short address`. Perbandingan dua cara ini adalah bahan analisis modul ini.

Prasyaratnya adalah M08: join, binding, endpoint, dan penghapusan NVS sebelum flash. Yang dibangun di sini adalah pemakaian `allowMultipleBinding(true)`, iterasi binding table melalui `getBoundDevices()`, pengalamatan per endpoint, perhitungan loss per node, serta pengamatan perilaku sistem saat satu node hilang. Semuanya dipakai lagi pada M10 ketika jenis anggota jaringan bertambah dengan hadirnya router, M12 pada komunikasi many-to-many di Thread, dan M16 ketika skalabilitas menjadi kriteria pemilihan protokol.

Peta modul blok Zigbee

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
07	802.15.4 telanjang
08	Jaringan Zigbee terbentuk: join, binding, cluster ON/OFF
09 (ini)	**Satu coordinator melayani banyak end device (binding table)**
10	Router menambah hop — routing multi-hop otomatis

Tabel 11.1: Peta modul blok Zigbee

Kontrak data lab ini. Identitas node di Zigbee adalah pasangan **`endpoint` ditambah `short address`** — bukan prefiks di dalam payload seperti M05 (`A:`, `B:`). Catat perbedaannya: identitas di sini dikelola **jaringan**, bukan aplikasi. Konsekuensinya muncul langsung di modul ini (lihat catatan `0xFFFF` di bagian Percobaan).

11.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Menskalakan Zigbee ke banyak end device

1. Membangun jaringan Zigbee satu coordinator dengan dua end device pada satu window join, dan menampilkan isi binding table beserta endpoint tiap node.
2. Mengirim perintah ke node tertentu berdasarkan `endpoint + short address` dan membuktikan node yang dituju bereaksi.
3. Menghitung packet loss **per node** (EP 10 dibanding EP 11) pada jarak yang sama dan pada jarak berbeda.
4. Menjelaskan perilaku coordinator ketika satu end device menghilang, berdasarkan log — termasuk apakah perintah tetap dikirim ke node yang sudah mati.

Kriteria keberhasilan

- ☐ Coordinator mencetak `Total 2 device.` pada daftar binding.
- ☐ Kedua lampu merespons pada siklus yang sama.
- ☐ Loss per node terukur terpisah (EP 10 dibanding EP 11).
- ☐ Skenario satu node dimatikan diuji dan perilakunya tercatat.

11.3 Dasar Teori (Secukupnya)

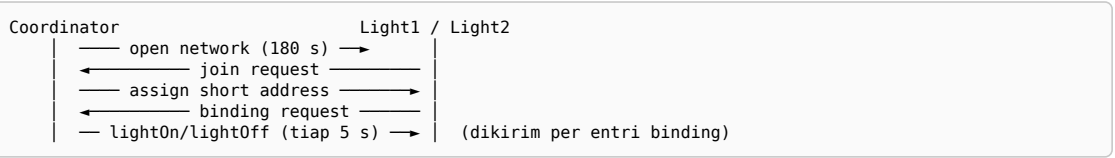
Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (tabel routing Zigbee, addressing mode APS, group addressing) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 11.2.

Istilah	Definisi kerja di lab ini
Coordinator (ZC)	Membentuk jaringan, mengelola PAN, mengizinkan join.
End Device (ZED)	Node akhir (child) rendah daya, tidak meneruskan trafik.
Join	Node bergabung ke jaringan; dibuka 180 detik setelah reboot (`setRebootOpenNetwork(180)`).
Binding table	Daftar tujuan yang dipegang switch; dibaca lewat `getBoundDevices()`.
Endpoint	Titik komunikasi aplikasi; Switch EP 5, Light1 EP 10, Light2 EP 11.
Cluster On/Off	Perintah standar; dipanggil per tujuan lewat `lightOn(ep, addr)`.
Short address	Alamat 16-bit node Zigbee, dicetak coordinator sebagai `0x%04X`.

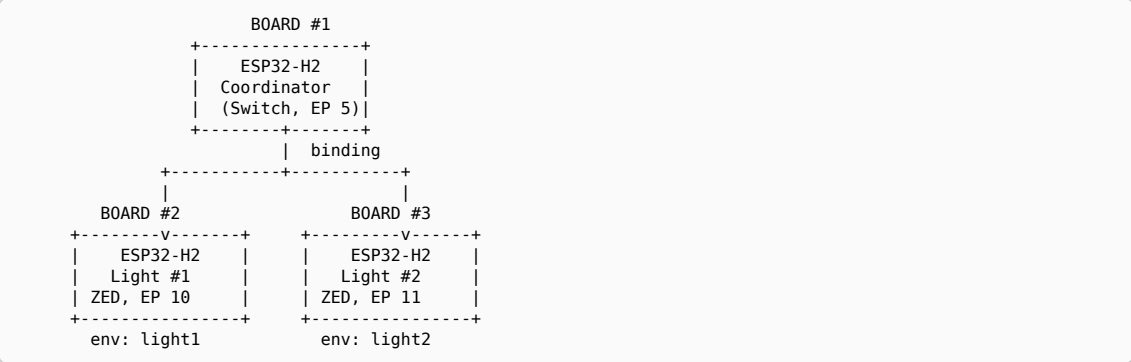
Tabel 11.2: Istilah kerja Modul 09

Mengapa endpoint kedua lampu harus berbeda? Karena binding table mengidentifikasi tujuan dari pasangan `endpoint + address`. Bila dua lampu memakai endpoint yang sama, entri binding menjadi ambigu dan perintah bisa menyasar. Endpoint adalah "nomor kamar" — dua kamar tidak boleh bernomor sama di satu daftar tujuan.

Sekuens protokol yang diamati



11.4 Topologi



Node	Board	Environment	Peran	Endpoint
Coordinator	ESP32-H2 DevKitM-1	`coordinator`	ZC dan switch (ZCZR)	EP 5
Light #1	ESP32-H2 DevKitM-1	`light1`	ZED dan light	EP 10
Light #2	ESP32-H2 DevKitM-1	`light2`	ZED dan light	EP 11

Tabel 11.3: Peran tiap node Modul 09

Ketiganya **ESP32-H2 DevKitM-1** dengan radio 802.15.4; peran ditentukan oleh `build_flags` dan tabel partisi, bukan oleh jenis board.

11.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan library `Zigbee` bawaan.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1, LED RGB bawaan	3
2	Kabel USB data	kabel data, bukan *charge-only*	3
3	PC/Laptop	PlatformIO Core/IDE, idealnya 3 port USB bebas	1
4	Power bank atau catu daya USB	untuk node yang dipindah saat uji jarak	2
5	Tabel partisi	`partitions_zczr.csv` (ZC), `partitions_ed.csv` (ED) — sudah ada di folder modul	—

Tabel 11.4: Alat dan bahan Modul 09

11.6 Kode Program

Tiga environment (Kode 11.1).

```
[env:coordinator]
build_src_filter = +<coordinator/*.cpp>
build_flags = -DZIGBEE_MODE_ZCZR -lesp_zb_api.zczr -lzboss_stack.zczr -lzboss_port.native
board_build.partitions = partitions_zczr.csv
upload_port = /dev/ttyACM0

[env:light1]
build_src_filter = +<light1/*.cpp>
build_flags = -DZIGBEE_MODE_ED -lesp_zb_api.ed -lzboss_stack.ed -lzboss_port.native
board_build.partitions = partitions_ed.csv
upload_port = /dev/ttyACM2

[env:light2]
build_src_filter = +<light2/*.cpp>
build_flags = -DZIGBEE_MODE_ED -lesp_zb_api.ed -lzboss_stack.ed -lzboss_port.native
board_build.partitions = partitions_ed.csv
upload_port = /dev/ttyACM4
```

Kode 11.1: Potongan `platformio.ini` — tiga environment Modul 09

```
[platformio]
default_envs = coordinator

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200

[env:coordinator]
build_src_filter = +<coordinator/*.cpp>
build_flags =
    -DZIGBEE_MODE_ZCZR
    -lesp_zb_api.zczr
    -lzboss_stack.zczr
    -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:light1]
build_src_filter = +<light1/*.cpp>
build_flags =
    -DZIGBEE_MODE_ED
    -lesp_zb_api.ed
    -lzboss_stack.ed
    -lzboss_port.native
board_build.partitions = partitions_ed.csv

[env:light2]
build_src_filter = +<light2/*.cpp>
build_flags =
    -DZIGBEE_MODE_ED
    -lesp_zb_api.ed
    -lzboss_stack.ed
    -lzboss_port.native
board_build.partitions = partitions_ed.csv
```

Kode 11.2: `platformio.ini` Modul 09 pada repositori

Tabel partisi `partitions_zczr.csv` dan `partitions_ed.csv` pada modul ini identik dengan milik Modul 08 (Kode 10.3 dan Kode 10.4), sehingga tidak diulang di sini.

```
// Minggu 9 — Zigbee Multi-Node: 1 Coordinator mengendalikan beberapa End Device
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Mode Zigbee ZCZR belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

ZigbeeSwitch zbSwitch = ZigbeeSwitch(5);

// Cetak parameter jaringan yang sedang dipakai. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
```

```
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Network Zigbee terbentuk:");
  Serial.println("  Peran          : Coordinator (ZC)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 5 (switch)");
}

void setup() {
  Serial.begin(115200);

  zbSwitch.setManufacturerAndModel("Espressif", "ZigbeeSwitch");
  zbSwitch.allowMultipleBinding(true);   // izinkan banyak light ter-bind

  Zigbee.addEndpoint(&zbSwitch);
  Zigbee.setRebootOpenNetwork(180);
  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin(ZIGBEE_COORDINATOR)) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  // Bagi coordinator, begin() baru kembali setelah network terbentuk (boot
  // pertama) atau dipulihkan dari NVS (reboot). JANGAN menunggu
  // Zigbee.connected() di sini: pada reboot dengan setRebootOpenNetwork(),
  // library Arduino core 3.3.x tidak pernah men-set flag itu untuk coordinator.
  printNetworkInfo();

  Serial.println("Menunggu light ter-binding (join dalam 180 detik)...");
  // Tunggu minimal 1 light ter-bind
  while (!zbSwitch.bound()) {
    delay(500);
  }
  delay(5000);  // beri waktu light lain join & bind

  Serial.println("Daftar device ter-bind:");
  std::list<zb_device_params_t *> bound = zbSwitch.getBoundDevices();
  for (auto d : bound) {
    Serial.printf(" - endpoint %d, short addr 0x%04X\n", d->endpoint, d->short_addr);
  }
  Serial.printf("Total %d device.\n", (int)bound.size());
}

void loop() {
  static unsigned long last = 0;
  static bool on = false;
  if (millis() - last > 5000) {
    last = millis();
    on = !on;
    std::list<zb_device_params_t *> bound = zbSwitch.getBoundDevices();
    for (auto d : bound) {
      if (on) {
        zbSwitch.lightOn(d->endpoint, d->short_addr);
        Serial.printf("-> Light 0x%04X ON\n", d->short_addr);
      } else {
        zbSwitch.lightOff(d->endpoint, d->short_addr);
        Serial.printf("-> Light 0x%04X OFF\n", d->short_addr);
      }
    }
  }
}
```

Kode 11.3: `src/coordinator/main.cpp` — coordinator dengan binding table

```ino
// Minggu 9 — Zigbee Multi-Node: Light #1 (end device)
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Mode Zigbee ED belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

uint8_t led = RGB_BUILTIN;

ZigbeeLight zbLight = ZigbeeLight(10);

void setLED(bool state) {
  digitalWrite(led, state);
  Serial.printf("Light1 %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang didapat saat join. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Info network:");
  Serial.println("  Peran          : End Device (ED)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X (diberikan coordinator)\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 10 (light)");
}

void setup() {
  Serial.begin(115200);
  pinMode(led, OUTPUT);
  digitalWrite(led, LOW);

  zbLight.setManufacturerAndModel("Espressif", "ZBLight1");
  zbLight.onLightChange(setLED);

  Zigbee.addEndpoint(&zbLight);

  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin()) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  Serial.println("Light1 menunggu join ke network...");
  while (!Zigbee.connected()) {
    delay(100);
  }
  Serial.println("Light1 tergabung ke network!");
  printNetworkInfo();
}

void loop() {
  delay(100);
}
```

Kode 11.4: `src/light1/main.cpp` — Light #1 pada endpoint 10

```ino
// Minggu 9 — Zigbee Multi-Node: Light #2 (end device)
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Mode Zigbee ED belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

uint8_t led = RGB_BUILTIN;
```

```
ZigbeeLight zbLight = ZigbeeLight(11);

void setLED(bool state) {
  digitalWrite(led, state);
  Serial.printf("Light2 %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang didapat saat join. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Info network:");
  Serial.println("  Peran          : End Device (ED)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X (diberikan coordinator)\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 11 (light)");
}

void setup() {
  Serial.begin(115200);
  pinMode(led, OUTPUT);
  digitalWrite(led, LOW);

  zbLight.setManufacturerAndModel("Espressif", "ZBLight2");
  zbLight.onLightChange(setLED);

  Zigbee.addEndpoint(&zbLight);

  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin()) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  Serial.println("Light2 menunggu join ke network...");
  while (!Zigbee.connected()) {
    delay(100);
  }
  Serial.println("Light2 tergabung ke network!");
  printNetworkInfo();
}

void loop() {
  delay(100);
}
```

Kode 11.5: `src/light2/main.cpp` — Light #2 pada endpoint 11

11.6.1 Channel antar Kelompok (wajib kelas paralel)

Seperti diperkenalkan pada Modul 08, setiap kelompok memakai **channel operasi yang berbeda** supaya node yang baru di-`erase` tidak salah join ke coordinator kelompok lain saat window join terbuka bersamaan, dan supaya angka loss serta latency tidak tercemar trafik kelompok tetangga pada channel yang sama.

Channel ditentukan coordinator saat membentuk jaringan; ketiga berkas pada listing di atas (Kode 11.3, Kode 11.4, Kode 11.5) memakai mask yang sama supaya kedua light hanya dapat menemukan jaringan kelompoknya sendiri. Jatah untuk enam kelompok ada pada Tabel 11.5.

Kelompok	Channel	Baris yang disisipkan sebelum `Zigbee.begin(...)`
1	15	`Zigbee.setPrimaryChannelMask(1UL << 15);`

Kelompok	Channel	Baris yang disisipkan sebelum `Zigbee.begin(...)`
2	20	`Zigbee.setPrimaryChannelMask(1UL << 20);`
3	25	`Zigbee.setPrimaryChannelMask(1UL << 25);`
4	11	`Zigbee.setPrimaryChannelMask(1UL << 11);`
5	17	`Zigbee.setPrimaryChannelMask(1UL << 17);`
6	22	`Zigbee.setPrimaryChannelMask(1UL << 22);`

Tabel 11.5: Jatah channel untuk enam kelompok — setiap kelompok memakai mask-nya sendiri

PERINGATAN
Pada coordinator baris ini menentukan channel jaringan yang dibentuk. Pada `src/light1/main.cpp` dan `src/light2/main.cpp` baris yang sama **bukan** memilih channel — kedua light tetap mengikuti channel coordinator — melainkan membatasi channel yang boleh dipindai.

TIP
Mengganti channel berarti mengganti jaringan yang tersimpan di NVS, jadi **jalankan `-t erase` di ketiga board** sebelum flash ulang. Setelah join, cocokkan `Channel` dan `Extended PAN ID` pada ketiga log: nilainya harus sama dengan coordinator kelompok sendiri. Bila sebuah light menampilkan channel kelompok lain, NVS-nya masih menyimpan keanggotaan lama — erase lalu join ulang.

11.7 Build dan Flash

Linux/macOS (bash):

```
for e in coordinator light1 light2; do pio run -d week09_zigbee_multinode -e $e -t erase; done
pio run -d week09_zigbee_multinode -e coordinator -t upload -t monitor
pio run -d week09_zigbee_multinode -e light1 -t upload      # dalam 180 s
pio run -d week09_zigbee_multinode -e light2 -t upload      # dalam 180 s
```

Windows (PowerShell):

```
$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'   # sekali per sesi terminal
pio run -e coordinator -t erase  --upload-port COM5    # port contoh; ganti
pio run -e light1      -t erase  --upload-port COM11   # sesuai pio device list
pio run -e light2      -t erase  --upload-port COM12
pio run -e coordinator -t upload --upload-port COM5
pio run -e light1      -t upload --upload-port COM11   # dalam 180 s
pio run -e light2      -t upload --upload-port COM12   # dalam 180 s
```

CATATAN
Prefix `$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'` mencegah `pio run` **menggantung tanpa batas** saat upload/erase di Windows — progress bar esptool memakai karakter Unicode (▉, ▒) yang tidak terwakili codepage default Windows. Setel ulang setiap membuka terminal baru. Erase cukup dijalankan sekali sebelum flash pertama, atau saat berganti channel/kelompok; mengulang percobaan dengan firmware yang sama tidak perlu erase lagi.

PENTING
Pilih port USB-to-UART, bukan USB native. Setiap board ESP32-H2 muncul sebagai **dua** port serial: jembatan USB-to-UART CH343 (`1a86:55d3`) dan USB-Serial/JTAG bawaan chip (`303a:1001`). Proses flash pada lab ini memakai **jembatan UART**, karena jalur itulah yang tersambung ke rangkaian *auto program* (DTR→IO9, RTS→EN) sehingga board masuk mode download tanpa menekan tombol. Pada Linux keduanya berselang-seling: port **genap** adalah UART, port **ganjil** adalah USB native. Satu board memakai `/dev/ttyACM0`, dua board `/dev/ttyACM0` dan `/dev/ttyACM2`, tiga board `/dev/ttyACM0`, `/dev/ttyACM2`, dan `/dev/ttyACM4`. Verifikasi dengan `pio device list` dan pilih port ber-Hardware ID `1A86:55D3`.

Pre-flight checklist

- [ ] `pio device list` dijalankan, tiga port dicatat dan diisikan di atas.
- [ ] Channel kelompok disetel pada ketiga berkas dan berbeda dari kelompok lain — lihat Tabel 11.5.
- [ ] **Hapus NVS ketiga board** (`-t erase`) sebelum flash pertama.
- [ ] Flash `coordinator` (ZCZR, `partitions_zczr.csv`).
- [ ] Flash `light1` dan `light2` (ED, `partitions_ed.csv`) **dalam window 180 detik**.
- [ ] Serial Monitor 115200 baud pada tiap board.
- [ ] Tabel pencatat jarak, RSSI, dan latency siap.

11.8 Percobaan

11.8.1 EXP-01 — Pembentukan Jaringan dan Join Ganda

Nyalakan coordinator lebih dulu. Jaringan terbuka 180 detik setelah reboot; dalam window ini nyalakan `light1` dan `light2` agar join dan binding.

```
[ZC reboot] ──► open network 180 s ──► [ZED1 join] [ZED2 join] ──► bound
```

Data capture

Parameter	Hasil
Waktu join Light1 (detik)	
Waktu join Light2 (detik)	
Status LED saat join	
Apakah keduanya masuk dalam satu window?	

Tabel 11.6: Lembar pengamatan EXP-01

CHECKPOINT

Kedua light mencetak `tergabung ke network!`. Jika hanya satu, window join sudah habis untuk yang kedua — reboot coordinator (window terbuka lagi) lalu ulangi. Jangan lanjut dengan satu node saja.

11.8.2 EXP-02 — Binding Table dan Kontrol Otomatis

Setelah minimal satu light ter-bind, coordinator menunggu 5 detik tambahan (agar light kedua sempat join dan bind), mencetak daftar device ter-bind, lalu men-toggle semua light setiap 5 detik.

```
[ZC] getBoundDevices() ──► endpoint + short addr
[ZC] lightOn(ep, addr) / lightOff(ep, addr)  tiap 5 s
```

Expected output — Coordinator (nilai channel, PAN ID, dan alamat dari uji di board; di board Anda akan berbeda)

```
Network Zigbee terbentuk:
  Peran          : Coordinator (ZC)
  Channel        : 26
  PAN ID         : 0x1890
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x0000
  IEEE address   : 74:4D:BD:FF:FE:61:E6:2C
  Endpoint       : 5 (switch)
Menunggu light ter-binding (join dalam 180 detik)...
Daftar device ter-bind:
 - endpoint 10, short addr 0xFFFF
 - endpoint 11, short addr 0xFFFF
Total 2 device.
-> Light 0xFFFF ON
-> Light 0xFFFF ON
-> Light 0xFFFF OFF
-> Light 0xFFFF OFF
```

CATATAN

Satu atau kedua `short addr` di daftar bound sering tercetak `0xFFFF` (pada uji ini keduanya), padahal short address asli kedua light terlihat di blok info network masing-masing. Itu **bukan** kegagalan binding — lihat catatan pada bagian Verifikasi Hardware di bawah.

Expected output — Light1

```
Light1 menunggu join ke network...
Light1 tergabung ke network!
Info network:
  Peran          : End Device (ED)
  Channel        : 26
  PAN ID         : 0x1890
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x82C3 (diberikan coordinator)
  IEEE address   : 74:4D:BD:FF:FE:61:E8:C1
  Endpoint       : 10 (light)
Light1 ON
Light1 OFF
```

Expected output — Light2

```
Light2 menunggu join ke network...
Light2 tergabung ke network!
Info network:
  Peran          : End Device (ED)
  Channel        : 26
  PAN ID         : 0x1890
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x127F (diberikan coordinator)
  IEEE address   : 74:4D:BD:FF:FE:61:F3:97
  Endpoint       : 11 (light)
Light2 ON
Light2 OFF
```

Info network ketiga board harus sama: **Channel, PAN ID, dan Extended PAN ID** (di sini 26, `0x1890`, dan IEEE address coordinator). Short address tiap light berbeda dan **dibagikan coordinator** saat join. Padanan XBee: `Short address` = MY, `IEEE address` = SH + SL, `Channel` = CH, `PAN ID` = OI, `Extended PAN ID` = OP — lihat tabel padanan XBee di Modul 08.

BUKA ABSTRAKSINYA

`getBoundDevices()` mengembalikan `std::list` berisi `zb_device_params_t`. Cetak **seluruh** field struct itu (bukan hanya endpoint dan short address) dan cocokkan dengan `IEEE address` (MAC 64-bit) yang dicetak tiap light pada blok `Info network`. Jawab: entri mana yang benar-benar unik dan stabil — short address atau alamat IEEE?

CHECKPOINT

Baris `Total 2 device.` muncul, dan setelah itu ada **dua** baris `-> Light 0x....` untuk tiap siklus ON dan tiap siklus OFF. Jika hanya satu baris per siklus, binding kedua gagal.

11.8.3 EXP-03 — Jarak dan Kehilangan Node

1. Letakkan Light2 pada jarak bertambah (1, 3, 5, lalu 10 m) dari coordinator; amati apakah perintah tetap diterima sementara Light1 tetap dekat.
2. Matikan (cabut daya) Light1 saat sistem berjalan; amati apakah coordinator **masih mencetak** `-> Light 0xXXXX ON` untuk node yang sudah mati.
3. Nyalakan kembali Light1 setelah window 180 detik berlalu; catat apakah dapat kembali sendiri dan apa yang perlu dilakukan.

Data capture

Parameter	Hasil
Success Light2 pada 10 m	
Apakah loss Light1 terpengaruh oleh Light2 yang jauh?	
Perilaku coordinator saat Light1 mati	
Light1 kembali otomatis? (ya/tidak) beserta alasan	

Tabel 11.7: Lembar pengamatan EXP-03

CHECKPOINT

Praktikan dapat menjelaskan mengapa coordinator tetap mengirim perintah ke node yang sudah mati (petunjuk: binding table adalah daftar statis, bukan daftar node yang sedang hidup). Ini temuan penting untuk desain sistem nyata.

11.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 3 × **ESP32-H2 DevKitM-1** (flash di-erase lebih dulu), capture 70 detik.

```
# Coordinator (ESP32-H2, ZCZR)
[0.401] Menunggu light ter-binding (join dalam 180 detik)...
[5.409] Daftar device ter-bind:
[5.409]  - endpoint 10, short addr 0xFFFF
[5.409]  - endpoint 11, short addr 0x1DA3
[5.409] Total 2 device.
[5.409] -> Light 0xFFFF ON
[5.409] -> Light 0x1DA3 ON

# Light1 (ESP32-H2, ED)            # Light2 (ESP32-H2, ED)
[0.602] Light1 tergabung ...       [3.208] Light2 tergabung ...
[5.411] Light1 ON                  [5.411] Light2 ON
[10.421] Light1 OFF                [10.419] Light2 OFF
```

Parameter	Hasil terukur
Device ter-bind terdeteksi	2 (EP 10 dan EP 11)
Perintah dikirim per light	14
Aksi terjadi di Light1 dan Light2	14 / 14 (0 % loss)

Parameter	Hasil terukur
Waktu join Light1 dan Light2	0,6 s dan 3,2 s

Tabel 11.8: Hasil verifikasi hardware Modul 09

> **CATATAN**
> **Short addr `0xFFFF` itu normal.** Entri binding yang dibuat lewat alamat IEEE (bukan alamat pendek) disimpan library dengan `short_addr = 0xFFFF`. `lightOn(ep, 0xFFFF)` tetap sampai ke node yang benar — buktinya Light1 tetap menyala. Yang perlu dicatat pada laporan adalah jumlah device terbind dan keberhasilan aksinya, bukan nilai alamatnya.

11.10 Pengukuran

Tabel jarak (geser Light2, Light1 tetap di 1 m; amati LED dan Serial Monitor).

Jarak Light2	RSSI	Latency	Success Light2	Success Light1
1 m			… / 10	… / 10
3 m			… / 10	… / 10
5 m			… / 10	… / 10
10 m			… / 10	… / 10
15 m			… / 10	… / 10

Tabel 11.9: Lembar pengukuran jarak Modul 09

Tabel per-node (jarak tetap 3 m, amati 20 perintah).

Node	RSSI	Paket diterima	Loss (%)
Light1 (EP 10)		… / 20	
Light2 (EP 11)		… / 20	

Tabel 11.10: Lembar pengukuran per-node Modul 09

Bandingkan dengan M05. Isi Tabel 11.11 memakai data BLE multi-node.

Aspek	BLE bintang (M05)	Zigbee multi-node (M09)
Cara pusat membedakan node		
Loss node dekat saat node lain jauh		
Perilaku saat satu node mati		
Batas jumlah node (perkiraan beserta alasan)		

Tabel 11.11: Tabel pembanding BLE bintang dan Zigbee multi-node

11.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Berapa waktu rata-rata proses join dari menyala hingga pesan `tergabung ke network`?
2. Apakah kedua light menerima perintah ON/OFF pada siklus yang sama? Buktikan dari urutan baris log coordinator.
3. Bagaimana pengaruh jarak terhadap success rate perintah ON/OFF, dan apakah node yang dekat ikut terdampak?
4. Apa yang terjadi di coordinator ketika salah satu light dimatikan? Masihkah perintah dikirim ke node itu, dan apa implikasinya untuk sistem nyata?
5. Mengapa endpoint light1 dan light2 harus berbeda (10 dibanding 11)? Apa yang akan terjadi jika sama?

11.12 Concept Check

1. Apa fungsi `allowMultipleBinding(true)` pada switch coordinator?
2. Apa yang dimaksud window join 180 detik pada `setRebootOpenNetwork(180)`, dan mengapa tidak dibuka selamanya?
3. Jelaskan perbedaan peran ZC dan ZED dalam topologi multi-node ini.
4. Bagaimana coordinator mengidentifikasi setiap light secara unik (endpoint ditambah short address)?
5. Apa keuntungan multi-binding dibanding broadcast ke semua node?

11.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Menambah node dan menghitung loss per endpoint

CH-1 — Node ketiga Tambahkan **light3** (endpoint 12): salin `src/light2`, ubah `ZigbeeLight(11)` menjadi `ZigbeeLight(12)` dan teks `Light2` menjadi `Light3`, lalu tambahkan env `light3` di `platformio.ini`. Amati `Total 3 device.` pada coordinator dan periksa apakah interval siklus bergeser.

CH-2 — Loss per node otomatis Catat jumlah perintah TX (`-> Light 0xXXXX ON/OFF`) dibanding jumlah `LightX ON/OFF` yang diterima tiap node selama 2 menit. Contoh: TX = 24, RX = 22, sehingga loss = (24 − 22)/24 × 100 % = 8,33 %. Sajikan terpisah per endpoint.

Level 3 — Tantangan · Kontrol selektif dan deteksi node hilang

CH-3 — Kontrol selektif Ubah coordinator agar menyalakan Light1 dan mematikan Light2 pada saat yang sama (bukan toggle serentak). Ini membuktikan perintah benar-benar dialamatkan per node, bukan disiarkan.

CH-4 — Deteksi node hilang Tambahkan mekanisme di coordinator untuk menandai node yang tidak merespons, misalnya menghitung berapa siklus berturut-turut tanpa laporan balik dari CH-2 Modul 08. Cetak `Node 0xXXXX tidak merespons` setelah 3 siklus.

11.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (Zigbee multi-node, ZC dan ZED, binding table, endpoint, cluster).
3. Konfigurasi — env `coordinator`, `light1`, `light2`, endpoint 5, 10, 11, serta interval 5 detik.

4. Hasil eksperimen — log join, daftar binding, log ketiga board (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — tabel jarak, tabel per-node, dan tabel perbandingan dengan M05.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-2.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.

12. Modul 10 — Zigbee Mesh: Routing Multi-Hop

IoT COMMUNICATION LAB · MODUL 10

Route Through the Mesh — Zigbee Mesh: Routing Multi-Hop

ESP32-H2 · Zigbee · ZC ke ZR ke ZED · level Advanced · 3 × 50 menit · folder kode `week10_zigbee_mesh`

PENGANTAR MODUL · Gambaran Umum

Modul 10 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat lanjut. Misinya membuktikan router benar-benar mengangkut trafik milik node lain, sekaligus mengukur harga hop keduanya. Percobaan berjalan sebagai mesh multi-hop dari coordinator ke router lalu ke end device, diamati melalui tiga terminal Serial Monitor pada 115200 baud dengan LED RGB bawaan sebagai penanda visual.

12.1 Pendahuluan

M06 sudah memperkenalkan hop — tetapi jalurnya ditulis sendiri di dalam kode. Di sini jalur ditentukan **stack**: end device memilih parent terbaik sendiri, dan router meneruskan trafik tanpa satu baris kode penerusan pun di sketch. Membandingkan dua pendekatan ini (relay manual M06 dibanding routing otomatis M10) adalah inti analisis modul ini, dan bekalnya dipakai lagi saat Thread melakukan hal serupa di atas IPv6 (M12).

Prasyaratnya ada dua: M09 untuk binding table, multi-node, dan penghapusan NVS; serta M06 untuk konsep hop dan loss per hop. Yang dibangun di sini adalah peran Router (ZR), pemilihan parent secara otomatis, routing multi-hop, perbandingan latency satu hop dengan dua hop, dan pengenalan status orphan. Semuanya dipakai lagi pada M12 ketika mesh Thread menambahkan role election, M13 ketika gateway berperan sebagai tepi jaringan, dan M16 ketika jangkauan mesh menjadi kriteria pemilihan protokol.

Peta modul blok Zigbee (penutup blok)

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
07	802.15.4 telanjang
08	Jaringan Zigbee terbentuk: join, binding
09	Satu coordinator, banyak end device (binding table)
10 (ini)	**Router meneruskan trafik — jangkauan melampaui satu hop**
11	Pindah ke Thread: IPv6 di atas radio 802.15.4 yang sama

Tabel 12.1: Peta modul blok Zigbee

Kontrak data lab ini. Ukur **latency 1 hop dan 2 hop** pada modul ini. Angka itu adalah pembanding langsung untuk latency relay manual M06 dan untuk mesh Thread M12 — tiga cara berbeda menyelesaikan masalah yang sama, di radio yang sama.

12.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membuktikan dan mengukur routing multi-hop Zigbee

1. Membangun jaringan Zigbee tiga peran (ZC, ZR, ZED) dan menunjukkan dari log bahwa ketiganya berada dalam satu jaringan.
2. Membuktikan end device mencapai coordinator **melalui** router pada formasi garis, dengan cara memutus jalur langsung dan mengamati akibatnya.
3. Mengukur selisih latency 1 hop (formasi dekat) dan 2 hop (formasi garis) lalu menyajikannya sebagai angka.
4. Menjelaskan status *orphan* pada end device saat router dimatikan, serta apakah dan bagaimana jaringan memulihkan diri.

Kriteria keberhasilan

- ☐ `Total device ter-bind: 2` tercetak di coordinator.
- ☐ EndLight tetap merespons pada formasi garis (ZC jauh dari ZED).
- ☐ Selisih latency 1 hop dan 2 hop terukur dan tercatat.
- ☐ Skenario router dimatikan diuji, perilaku ZED tercatat.

12.3 Dasar Teori (Secukupnya)

Teori dibatasi pada apa yang dipakai di percobaan. Pembahasan mendalam (algoritma routing AODV Zigbee, tabel routing, many-to-one routing) berada di buku teori terpisah. Istilah kerja dirangkum pada Tabel 12.2.

Istilah	Definisi kerja di lab ini
Coordinator (ZC)	Membentuk jaringan, membuka window join 180 detik, sumber perintah ON/OFF.
Router (ZR)	Node penuh yang **meneruskan paket node lain** (`Zigbee.begin(ZIGBEE_ROUTER)`); juga punya aplikasi lampu (EP 10).
End Device (ZED)	Child yang bergabung lewat parent (bisa router), tidak meneruskan trafik.
Parent selection	ZED memilih parent dengan kualitas tautan terbaik — bisa ZC, bisa ZR.
Mesh	Jalur alternatif otomatis; EndLight bisa mencapai ZC lewat ZR (2 hop) atau langsung (1 hop).
Orphan	Status ZED yang kehilangan parent dan belum menemukan pengganti.
Endpoint/Cluster	EP 5 (switch), EP 10 (router light), EP 11 (ED light), semuanya pada cluster On/Off.

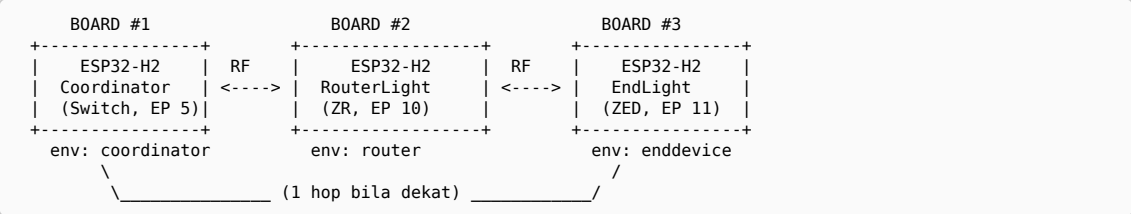
Tabel 12.2: Istilah kerja Modul 10

Router bukan sekadar "node yang juga menyala". Perbedaan sesungguhnya: router **selalu mendengarkan** (tidak tidur) dan menyimpan tabel routing. Itulah mengapa ZR memakai firmware ZCZR (lebih besar, partisi berbeda) dan mengapa ZR tidak cocok untuk node baterai. Catat konsekuensi daya ini — ia menjadi salah satu argumen di M16.

Sekuens protokol yang diamati

```
ZC ──(lightOn/lightOff)──► ZR (EP10: nyala)
                            │
                            └── relay ──► ZED (EP11: nyala)
```

12.4 Topologi



Node	Board	Environment	Peran	Endpoint
Coordinator	ESP32-H2 DevKitM-1	`coordinator`	ZC dan switch (ZCZR)	EP 5
RouterLight	ESP32-H2 DevKitM-1	`router`	ZR dan light (ZCZR)	EP 10
EndLight	ESP32-H2 DevKitM-1	`enddevice`	ZED dan light (ED)	EP 11

Tabel 12.3: Peran tiap node Modul 10

Ketiganya **ESP32-H2 DevKitM-1**. Router memakai firmware ZCZR (`partitions_zczr.csv`) sedangkan end device memakai ED (`partitions_ed.csv`) — satu-satunya perbedaan perangkat keras adalah peran yang diberikan, bukan chip.

Rencana penempatan: ZC, jarak, ZR, jarak, ZED, dalam satu garis. Formasi meja (semua kurang dari 1 m) hanya untuk pemanasan; ia **tidak** membuktikan multi-hop.

12.5 Alat yang Digunakan

Modul ini dijalankan di atas ESP32-H2 (Arduino core 3.x) dengan library `Zigbee` bawaan.

No	Peralatan	Spesifikasi	Jumlah
1	Board ESP32-H2	DevKitM-1, LED RGB bawaan	3
2	Kabel USB data	kabel data, bukan *charge-only*	3
3	PC/Laptop	PlatformIO Core/IDE	1
4	Power bank atau catu daya USB	wajib — ZC dan ZED harus bisa dijauhkan	3
5	Ruang uji	lorong atau ruang panjang untuk formasi garis	—

Tabel 12.4: Alat dan bahan Modul 10

12.6 Kode Program

Tiga peran, dua tabel partisi (Kode 12.1).

```
[env:coordinator]
build_flags = -DZIGBEE_MODE_ZCZR -lesp_zb_api.zczr -lzboss_stack.zczr -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:router]                ; ZR juga memakai firmware ZCZR
build_flags = -DZIGBEE_MODE_ZCZR -lesp_zb_api.zczr -lzboss_stack.zczr -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:enddevice]
build_flags = -DZIGBEE_MODE_ED -lesp_zb_api.ed -lzboss_stack.ed -lzboss_port.native
board_build.partitions = partitions_ed.csv
```

Kode 12.1: Potongan `platformio.ini` — tiga peran Modul 10

```
[platformio]
default_envs = coordinator

[env]
; pioarduino fork: platform resmi PlatformIO belum menyediakan board ESP32-H2
platform = https://github.com/pioarduino/platform-espressif32/releases/download/55.03.311/platform-espressif
32.zip
framework = arduino
board = esp32-h2-devkitm-1
monitor_speed = 115200

[env:coordinator]
build_src_filter = +<coordinator/*.cpp>
build_flags =
    -DZIGBEE_MODE_ZCZR
    -lesp_zb_api.zczr
    -lzboss_stack.zczr
    -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:router]
build_src_filter = +<router/*.cpp>
build_flags =
    -DZIGBEE_MODE_ZCZR
    -lesp_zb_api.zczr
    -lzboss_stack.zczr
    -lzboss_port.native
board_build.partitions = partitions_zczr.csv

[env:enddevice]
build_src_filter = +<enddevice/*.cpp>
build_flags =
    -DZIGBEE_MODE_ED
    -lesp_zb_api.ed
    -lzboss_stack.ed
    -lzboss_port.native
board_build.partitions = partitions_ed.csv
```

Kode 12.2: `platformio.ini` Modul 10 pada repositori

Tabel partisi `partitions_zczr.csv` dan `partitions_ed.csv` pada modul ini identik dengan milik Modul 08 (Kode 10.3 dan Kode 10.4).

```
// Minggu 10 — Zigbee Mesh: Coordinator mengendalikan lampu lewat Router & ED
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Mode Zigbee ZCZR belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

ZigbeeSwitch zbSwitch = ZigbeeSwitch(5);

// Cetak parameter jaringan yang sedang dipakai. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Network Zigbee terbentuk:");
  Serial.println("  Peran          : Coordinator (ZC)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 5 (switch)");
}

void setup() {
  Serial.begin(115200);
```

```
  zbSwitch.setManufacturerAndModel("Espressif", "ZigbeeSwitch");
  zbSwitch.allowMultipleBinding(true);

  Zigbee.addEndpoint(&zbSwitch);
  Zigbee.setRebootOpenNetwork(180);
  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin(ZIGBEE_COORDINATOR)) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  // Bagi coordinator, begin() baru kembali setelah network terbentuk (boot
  // pertama) atau dipulihkan dari NVS (reboot). JANGAN menunggu
  // Zigbee.connected() di sini: pada reboot dengan setRebootOpenNetwork(),
  // library Arduino core 3.3.x tidak pernah men-set flag itu untuk coordinator.
  printNetworkInfo();

  Serial.println("Menunggu router & end device ter-binding...");
  while (!zbSwitch.bound()) {
    delay(500);
  }
  delay(8000);  // beri waktu router dan ED join & bind

  std::list<zb_device_params_t *> bound = zbSwitch.getBoundDevices();
  Serial.printf("Total device ter-bind: %d\n", (int)bound.size());
  for (auto d : bound) {
    Serial.printf(" - endpoint %d, short addr 0x%04X\n", d->endpoint, d->short_addr);
  }
}

void loop() {
  static unsigned long last = 0;
  static bool on = false;
  if (millis() - last > 5000) {
    last = millis();
    on = !on;
    std::list<zb_device_params_t *> bound = zbSwitch.getBoundDevices();
    for (auto d : bound) {
      if (on) {
        zbSwitch.lightOn(d->endpoint, d->short_addr);
        Serial.printf("-> 0x%04X ON\n", d->short_addr);
      } else {
        zbSwitch.lightOff(d->endpoint, d->short_addr);
        Serial.printf("-> 0x%04X OFF\n", d->short_addr);
      }
    }
  }
}
```

Kode 12.3: `src/coordinator/main.cpp` — coordinator sekaligus switch (EP 5)

```
// Minggu 10 — Zigbee Mesh: Router (lampu yang juga meneruskan trafik)
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Mode Zigbee ZCZR belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

uint8_t led = RGB_BUILTIN;

ZigbeeLight zbLight = ZigbeeLight(10);

void setLED(bool state) {
  digitalWrite(led, state);
  Serial.printf("RouterLight %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang didapat saat join. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
```

```
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Info network:");
  Serial.println("  Peran          : Router (ZR)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X (diberikan coordinator)\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 10 (light)");
}

void setup() {
  Serial.begin(115200);
  pinMode(led, OUTPUT);
  digitalWrite(led, LOW);

  zbLight.setManufacturerAndModel("Espressif", "ZBLightRouter");
  zbLight.onLightChange(setLED);

  Zigbee.addEndpoint(&zbLight);

  Zigbee.setPrimaryChannelMask(1UL << 15);

  // Peran ROUTER: bisa meneruskan trafik ke node lain (mesh)
  if (!Zigbee.begin(ZIGBEE_ROUTER)) {
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  Serial.println("Router menunggu join ke network...");
  while (!Zigbee.connected()) {
    delay(100);
  }
  Serial.println("Router tergabung (role=ROUTER).");
  printNetworkInfo();
}

void loop() {
  delay(100);
}
```

Kode 12.4: `src/router/main.cpp` — RouterLight (ZR, EP 10)

```
// Minggu 10 — Zigbee Mesh: End Device (child, dapat join lewat router)
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Mode Zigbee ED belum dipilih (periksa build_flags)"
#endif
#include "Zigbee.h"

uint8_t led = RGB_BUILTIN;

ZigbeeLight zbLight = ZigbeeLight(11);

void setLED(bool state) {
  digitalWrite(led, state);
  Serial.printf("EndLight %s\n", state ? "ON" : "OFF");
}

// Cetak parameter jaringan yang didapat saat join. Library Arduino hanya
// menulisnya ke log debug, jadi dibaca langsung dari stack ESP-Zigbee
// (wajib di dalam lock karena stack berjalan di task lain).
void printNetworkInfo() {
  esp_zb_ieee_addr_t extPanId, ieee;
  esp_zb_lock_acquire(portMAX_DELAY);
  uint8_t channel = esp_zb_get_current_channel();
  uint16_t panId = esp_zb_get_pan_id();
  uint16_t shortAddr = esp_zb_get_short_address();
  esp_zb_get_extended_pan_id(extPanId);
  esp_zb_get_long_address(ieee);
  esp_zb_lock_release();

  Serial.println("Info network:");
  Serial.println("  Peran          : End Device (ED)");
  Serial.printf("  Channel        : %u\n", channel);
  Serial.printf("  PAN ID         : 0x%04X\n", panId);
```

```
  Serial.printf("  Extended PAN ID: %s\n", ZigbeeCore::formatIEEEAddress(extPanId));
  Serial.printf("  Short address  : 0x%04X (diberikan coordinator)\n", shortAddr);
  Serial.printf("  IEEE address   : %s\n", ZigbeeCore::formatIEEEAddress(ieee));
  Serial.println("  Endpoint       : 11 (light)");
}

void setup() {
  Serial.begin(115200);
  pinMode(led, OUTPUT);
  digitalWrite(led, LOW);

  zbLight.setManufacturerAndModel("Espressif", "ZBLightED");
  zbLight.onLightChange(setLED);

  Zigbee.addEndpoint(&zbLight);

  Zigbee.setPrimaryChannelMask(1UL << 15);

  if (!Zigbee.begin()) {  // default = ZIGBEE_END_DEVICE
    Serial.println("Zigbee gagal start!");
    ESP.restart();
  }

  Serial.println("End device menunggu join (bisa lewat router)...");
  while (!Zigbee.connected()) {
    delay(100);
  }
  Serial.println("End device tergabung (role=END_DEVICE).");
  printNetworkInfo();
}

void loop() {
  delay(100);
}
```

Kode 12.5: `src/enddevice/main.cpp` — EndLight (ZED, EP 11)

12.6.1 Channel antar Kelompok (wajib kelas paralel)

Satu ruangan praktikum biasanya menampung beberapa kelompok, sehingga berdiri beberapa jaringan Zigbee sekaligus. Satu node tetap hanya menjadi anggota satu jaringan; yang harus dicegah adalah dua hal: node yang baru di-`erase` dapat **salah join** ke coordinator kelompok lain bila beberapa window join terbuka bersamaan, dan beberapa jaringan pada channel yang sama saling berebut airtime sehingga angka loss serta latency — data utama modul ini — tercemar oleh tetangga.

Channel operasi ditentukan coordinator saat membentuk jaringan (parameter **SC**, lihat Modul 08), jadi nilainya disetel lewat `setPrimaryChannelMask()` sebelum `Zigbee.begin()` — pada **ketiga** berkas. Tiga listing di atas sudah memuat baris itu (Kode 12.3, Kode 12.4, Kode 12.5) dengan jatah Kelompok 1; setiap kelompok mengganti angka `15` sesuai Tabel 12.5.

Kelompok	Channel	Baris yang disisipkan sebelum `Zigbee.begin(...)`
1	15	`Zigbee.setPrimaryChannelMask(1UL << 15);`
2	20	`Zigbee.setPrimaryChannelMask(1UL << 20);`
3	25	`Zigbee.setPrimaryChannelMask(1UL << 25);`
4	11	`Zigbee.setPrimaryChannelMask(1UL << 11);`
5	17	`Zigbee.setPrimaryChannelMask(1UL << 17);`
6	22	`Zigbee.setPrimaryChannelMask(1UL << 22);`

Tabel 12.5: Jatah channel untuk enam kelompok — setiap kelompok memakai mask-nya sendiri

Mask Zigbee adalah satu bit per channel (`1UL << n` berarti **hanya** channel n). Mask default `0x07FFF800` mencakup channel 11–26 sehingga coordinator bebas memilih di antaranya; dengan jatah di Tabel 12.5, tiap kelompok mengunci satu channel saja.

PERINGATAN
Pada coordinator baris ini menentukan channel jaringan yang dibentuk. Pada `src/router/main.cpp` dan `src/enddevice/main.cpp` baris yang sama **bukan** memilih channel — ZR dan ZED tetap mengikuti channel coordinator — melainkan membatasi channel yang boleh dipindai, supaya keduanya hanya dapat menemukan jaringan kelompoknya sendiri.

TIP
Channel 15, 20, dan 25 adalah pilihan klasik Zigbee karena paling longgar terhadap Wi-Fi 2,4 GHz; 11, 17, dan 22 melengkapi sebaran agar antar kelompok terpisah minimal dua kanal. Mengganti channel berarti mengganti jaringan yang tersimpan di NVS, jadi **jalankan `-t erase` di ketiga board** sebelum flash ulang. Setelah join, cocokkan `Channel` dan `Extended PAN ID` pada ketiga log: `Channel` harus sesuai jatah kelompok dan `Extended PAN ID` harus IEEE address coordinator sendiri. Bila sebuah board menampilkan channel kelompok lain, NVS-nya masih menyimpan keanggotaan lama — erase lalu join ulang.

12.7 Build dan Flash

Urutan penting: ZC, lalu ZR, lalu ZED.

Linux/macOS (bash):

```
for e in coordinator router enddevice; do pio run -d week10_zigbee_mesh -e $e -t erase; done
pio run -d week10_zigbee_mesh -e coordinator -t upload -t monitor
pio run -d week10_zigbee_mesh -e router      -t upload    # dalam 180 s
pio run -d week10_zigbee_mesh -e enddevice   -t upload    # dalam 180 s
```

Windows (PowerShell):

```
$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'   # sekali per sesi terminal
pio run -e coordinator -t erase  --upload-port COM5    # port contoh; ganti
pio run -e router      -t erase  --upload-port COM11   # sesuai pio device list
pio run -e enddevice   -t erase  --upload-port COM12
pio run -e coordinator -t upload --upload-port COM5
pio run -e router      -t upload --upload-port COM11   # dalam 180 s
pio run -e enddevice   -t upload --upload-port COM12   # dalam 180 s
```

CATATAN
Prefix `$env:PYTHONIOENCODING='utf-8'; $env:PYTHONUTF8='1'` mencegah `pio run` **menggantung tanpa batas** saat upload/erase di Windows — progress bar esptool memakai karakter Unicode (▉, ▒) yang tidak terwakili codepage default Windows. Setel ulang setiap membuka terminal baru. Erase cukup dijalankan sekali sebelum flash pertama, atau saat berganti channel/kelompok; mengulang percobaan dengan firmware yang sama tidak perlu erase lagi.

Pre-flight checklist

- ☐ `pio device list` dijalankan, tiga port dicatat dan diisikan pada tiap env.
- ☐ Channel kelompok disetel pada ketiga berkas dan berbeda dari kelompok lain — lihat Tabel 12.5.
- ☐ **Hapus NVS ketiga board** (`-t erase`) sebelum flash pertama.
- ☐ Flash `coordinator` (ZCZR), `router` (ZCZR, mulai sebagai `ZIGBEE_ROUTER`), dan `enddevice` (ED) — semuanya dalam window 180 detik.
- ☐ Serial Monitor 115200 baud pada ketiga board.
- ☐ Formasi penempatan (meja dulu, lalu garis) sudah direncanakan.

12.8 Percobaan

12.8.1 EXP-01 — Join Bertingkat

Nyalakan coordinator, lalu router, lalu end device (semua dalam window 180 detik). Router join langsung ke ZC; end device memilih parent terbaik.

```
[ZC on] ──► open 180 s ──► [ZR join ke ZC] ──► [ZED join (parent = ZR/ZC)]
```

Data capture

Parameter	Hasil
Waktu join Router (detik)	
Waktu join End Device (detik)	
Total device ter-bind di ZC	
Role yang dicetak Router dan ZED	

Tabel 12.6: Lembar pengamatan EXP-01

CHECKPOINT

Router mencetak `role=ROUTER` dan end device mencetak `role=END_DEVICE`. Jika router mencetak `END_DEVICE`, `build_flags`-nya salah — perbaiki sebelum lanjut, karena tanpa ZR tidak ada hop kedua.

12.8.2 EXP-02 — Kontrol Multi-Hop

Coordinator menunggu binding (ditambah 8 detik), mencetak daftar device, lalu men-toggle semua device tiap 5 detik. Perintah ke EndLight diteruskan router bila ZED memilih ZR sebagai parent.

```
[ZC] ─── ON/OFF (5 s) ──► [ZR: RouterLight ON/OFF]
                │
                └─ relay ──► [ZED: EndLight ON/OFF]
```

Expected output — Coordinator (nilai channel, PAN ID, dan alamat dari uji di board; di board Anda akan berbeda)

```
Network Zigbee terbentuk:
  Peran          : Coordinator (ZC)
  Channel        : 21
  PAN ID         : 0x4D92
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x0000
  IEEE address   : 74:4D:BD:FF:FE:61:E6:2C
  Endpoint       : 5 (switch)
Menunggu router & end device ter-binding...
Total device ter-bind: 2
 - endpoint 10, short addr 0xFFFF
 - endpoint 11, short addr 0xBB28
-> 0xFFFF ON
-> 0xBB28 ON
-> 0xFFFF OFF
-> 0xBB28 OFF
```

Expected output — Router

```
Router menunggu join ke network...
Router tergabung (role=ROUTER).
Info network:
  Peran          : Router (ZR)
  Channel        : 21
  PAN ID         : 0x4D92
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0x33F4 (diberikan coordinator)
  IEEE address   : 74:4D:BD:FF:FE:61:E8:C1
  Endpoint       : 10 (light)
RouterLight ON
RouterLight OFF
```

Expected output — End device

```
End device menunggu join (bisa lewat router)...
End device tergabung (role=END_DEVICE).
Info network:
  Peran          : End Device (ED)
  Channel        : 21
  PAN ID         : 0x4D92
  Extended PAN ID: 74:4D:BD:FF:FE:61:E6:2C
  Short address  : 0xBB28 (diberikan coordinator)
  IEEE address   : 74:4D:BD:FF:FE:61:F3:97
  Endpoint       : 11 (light)
EndLight ON
EndLight OFF
```

Info network ketiga board harus sama: **Channel, PAN ID, dan Extended PAN ID** (di sini 21, `0x4D92`, dan IEEE address coordinator). Router juga mendapat short address dari coordinator, sama seperti end device. Short address end device (`0xBB28`) sama dengan yang tercetak di daftar bound coordinator; router tercetak `0xFFFF` di sana. Padanan XBee: `Short address` = MY, `IEEE address` = SH + SL, `Channel` = CH, `PAN ID` = OI, `Extended PAN ID` = OP — lihat tabel padanan XBee di Modul 08.

BUKA ABSTRAKSINYA

Cari di `src/router/main.cpp` baris kode yang **meneruskan** perintah ke end device. Baris itu tidak akan ditemukan: penerusan dikerjakan stack Zigbee, bukan aplikasi. Bandingkan dengan `src/nodeb/main.cpp` di Modul 06, tempat penerusan ditulis eksplisit. Tuliskan perbandingan itu — ia adalah jawaban pertanyaan analisis nomor 5.

CHECKPOINT

Dua baris `-> 0x....` muncul tiap siklus dan kedua LED berubah. Jika EndLight tidak ikut berubah padahal ter-bind, dekatkan dulu semuanya (formasi meja) sebelum mencoba formasi garis.

12.8.3 EXP-03 — Varian Topologi (inti modul)

1. **Formasi garis** (ZC, 5 m, ZR, 5 m, ZED): EndLight menerima perintah lewat router (2 hop). Amati apakah tetap ON/OFF sinkron.
2. **Formasi dekat**: letakkan ZED bersebelahan ZC; ZED mungkin memilih ZC sebagai parent (1 hop). Bandingkan latency nyala LED.
3. **Hilangkan router**: matikan ZR pada formasi garis; amati apakah ZED menjadi *orphan*, lalu apakah dapat re-join ke ZC bila jarak masih terjangkau.
4. Nyalakan ZR kembali dan catat waktu pemulihan.

Data capture

Parameter	Hasil
Latency EndLight, formasi dekat (1 hop)	
Latency EndLight, formasi garis (2 hop)	
Selisih latency 1 hop dan 2 hop	
Perilaku ZED saat ZR dimatikan	
Waktu pemulihan setelah ZR hidup lagi	

Tabel 12.7: Lembar pengamatan EXP-03

CHECKPOINT

Tersedia **dua angka latency** dari dua formasi berbeda, bukan satu. Tanpa keduanya, klaim "multi-hop berhasil" tidak bisa dibuktikan.

12.9 Verifikasi Hardware — Log Referensi

Dijalankan pada 3 × **ESP32-H2 DevKitM-1** (flash di-erase lebih dulu; ketiga board berdekatan, jadi ZED kemungkinan besar memilih ZC sebagai parent, yaitu 1 hop), capture 80 detik.

```
# Coordinator (ESP32-H2, ZCZR)
[0.401] Menunggu router & end device ter-binding...
[8.415] Total device ter-bind: 2
[8.415]  - endpoint 10, short addr 0xFFFF
[8.415]  - endpoint 11, short addr 0x727D
[8.415] -> 0xFFFF ON
[8.415] -> 0x727D ON

# Router (ESP32-H2, ZR)           # End device (ESP32-H2, ZED)
[0.401] Router tergabung          [3.206] End device tergabung
        (role=ROUTER).                    (role=END_DEVICE).
[8.416] RouterLight ON            [8.415] EndLight ON
[13.425] RouterLight OFF          [13.423] EndLight OFF
```

Parameter	Hasil terukur
Waktu join router dan end device	0,4 s dan 3,2 s
Semua node ter-bind	2 light (EP 10 dan EP 11) pada detik 8,4
Perintah dikirim per light	15
Aksi terjadi di RouterLight dan EndLight	15 / 15 (0 % loss)

Tabel 12.8: Hasil verifikasi hardware Modul 10

Formasi meja (semua node kurang dari 1 m) **belum membuktikan** routing multi-hop. Untuk itu jalankan EXP-03 formasi garis dan matikan jalur langsung ZC ke ZED.

12.10 Pengukuran

Tabel jarak ZC ke ZED (dengan ZR di tengah).

Jarak total	RSSI	Latency	Success
1 m			… / 10
3 m			… / 10
5 m			… / 10
10 m			… / 10
15 m			… / 10

Tabel 12.9: Lembar pengukuran jarak Modul 10

Tabel per-hop atau per-node (20 perintah).

Node atau Hop	RSSI	Paket diterima	Loss (%)
Router (EP 10, hop 1)		… / 20	
EndLight (EP 11, hop 2)		… / 20	

Tabel 12.10: Lembar pengukuran per-hop Modul 10

Tabel pembanding hop — wajib. Isi dari data sendiri.

Cara mencapai node jauh	Modul	Latency terukur	Siapa yang menentukan jalur
Relay manual (BLE)	M06		kode aplikasi
Routing otomatis (Zigbee)	M10		stack Zigbee

Tabel 12.11: Tabel pembanding relay manual dan routing otomatis

12.11 Analisis

Jawab berdasarkan tabel bagian Pengukuran.

1. Berapa lama waktu join router dan end device? Apakah urutan penyalaan memengaruhi keberhasilan?
2. Berdasarkan formasi dekat dibanding garis, kapan end device memilih router sebagai parent? Bagaimana hal itu dibuktikan dari log?
3. Bandingkan latency nyala EndLight pada 1 hop dan 2 hop; berapa selisihnya dan apakah sebanding dengan tambahan jangkauan?
4. Apa yang terjadi pada EndLight ketika router dimatikan? Sebutkan istilah status node tersebut dan berapa lama pemulihannya.
5. Bandingkan dengan relay manual M06: apa yang **diperoleh** dari routing otomatis, dan apa yang **hilang** (kendali, keterlihatan, ukuran firmware)?

12.12 Concept Check

1. Apa perbedaan utama mode build ZCZR dan ED (lihat `build_flags` di `platformio.ini`)?
2. Mengapa router memakai `partitions_zczr.csv` sedangkan end device `partitions_ed.csv`?
3. Bagaimana mesh Zigbee menemukan jalur baru jika salah satu router mati?
4. Jelaskan hubungan endpoint 5, 10, dan 11 dengan binding pada percobaan ini.
5. Apa konsekuensi daya dari node yang berperan sebagai router dibanding end device, dan apa artinya untuk node bertenaga baterai?

12.13 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Memperpanjang rantai dan mengukur loss per hop

CH-1 — Rantai dua router Tambahkan **router2** (salin `src/router`, tetap `ZigbeeLight(10)` tetapi ubah model string) dan susun rantai ZC, ZR1, ZR2, ZED. Amati berapa hop maksimum yang masih berfungsi dan berapa tambahan latency per hop.

CH-2 — Packet loss per hop Catat TX di ZC dan RX di tiap lampu selama 3 menit. Contoh: TX = 36, RX EndLight = 33, sehingga loss = (36 − 33)/36 × 100 % = 8,33 %. Sajikan terpisah untuk hop 1 dan hop 2.

Level 3 — Tantangan · Melacak parent dan menguji self-healing

CH-3 — Lacak parent Amati alamat parent end device dengan memindahkan ZED bertahap, lalu simpulkan pada jarak berapa jalur mulai di-router. Sajikan sebagai tabel jarak terhadap parent.

CH-4 — Uji self-healing Pada formasi garis, matikan ZR lalu ukur berapa lama ZED butuh untuk kembali (baik lewat ZC langsung maupun setelah ZR hidup lagi). Bandingkan dengan perilaku relay manual M06 yang tidak pulih sendiri.

12.14 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas (mesh Zigbee, ZR sebagai relayer, parent selection, orphan).
3. Konfigurasi — env `coordinator`, `router`, `enddevice`, endpoint 5, 10, 11, interval 5 detik, dan window 180 s.
4. Hasil eksperimen — log ketiga node, skema formasi (meja dan garis), serta checkpoint.
5. Data pengukuran — tabel jarak, tabel per-hop, dan tabel pembanding hop M06 dengan M10.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-3.
8. Kesimpulan yang ditulis sendiri berdasarkan hasil pengujian.